ECE 590
Neural-Network-Based Large Language Models

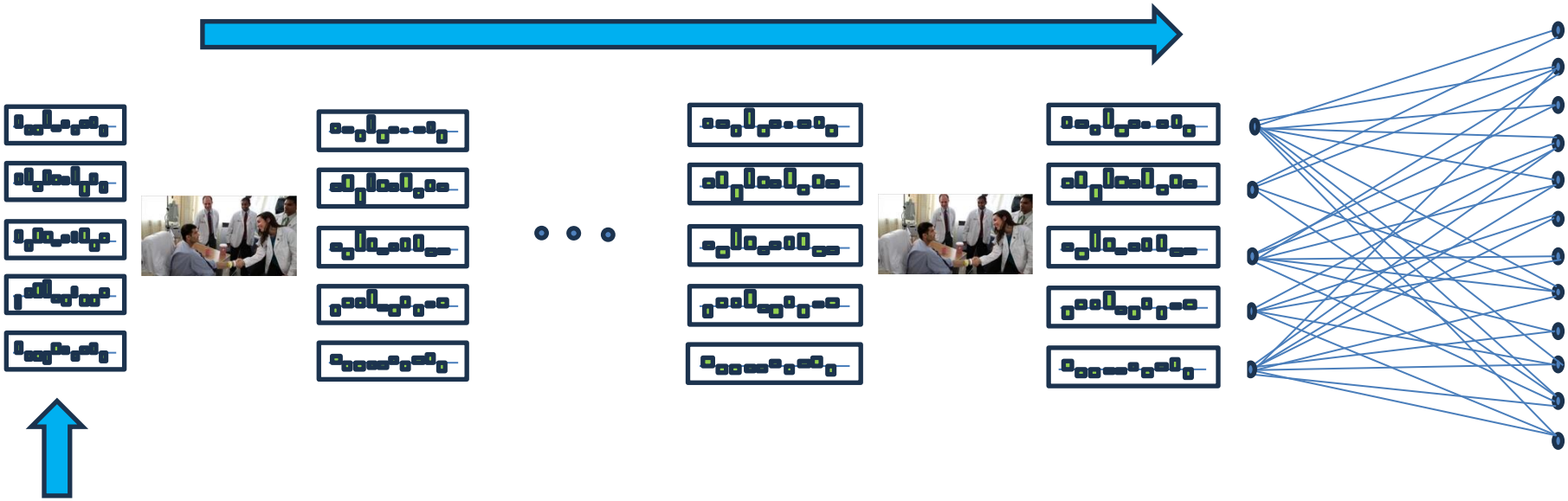
Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 8
02/03/2026

Generative Pre-trained Transformer (GPT)

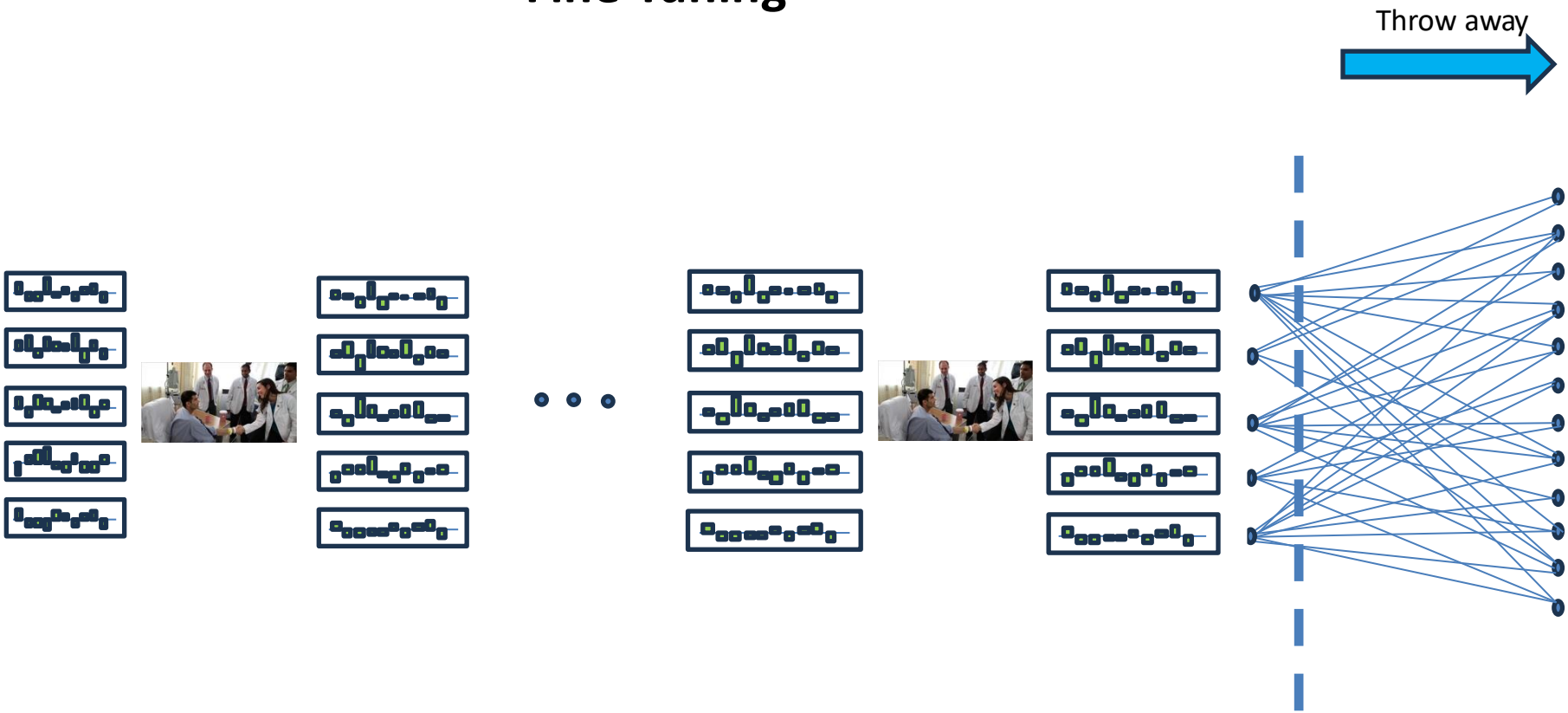
L layers of context-imposition, defined by word (“person”) similarity & proximity



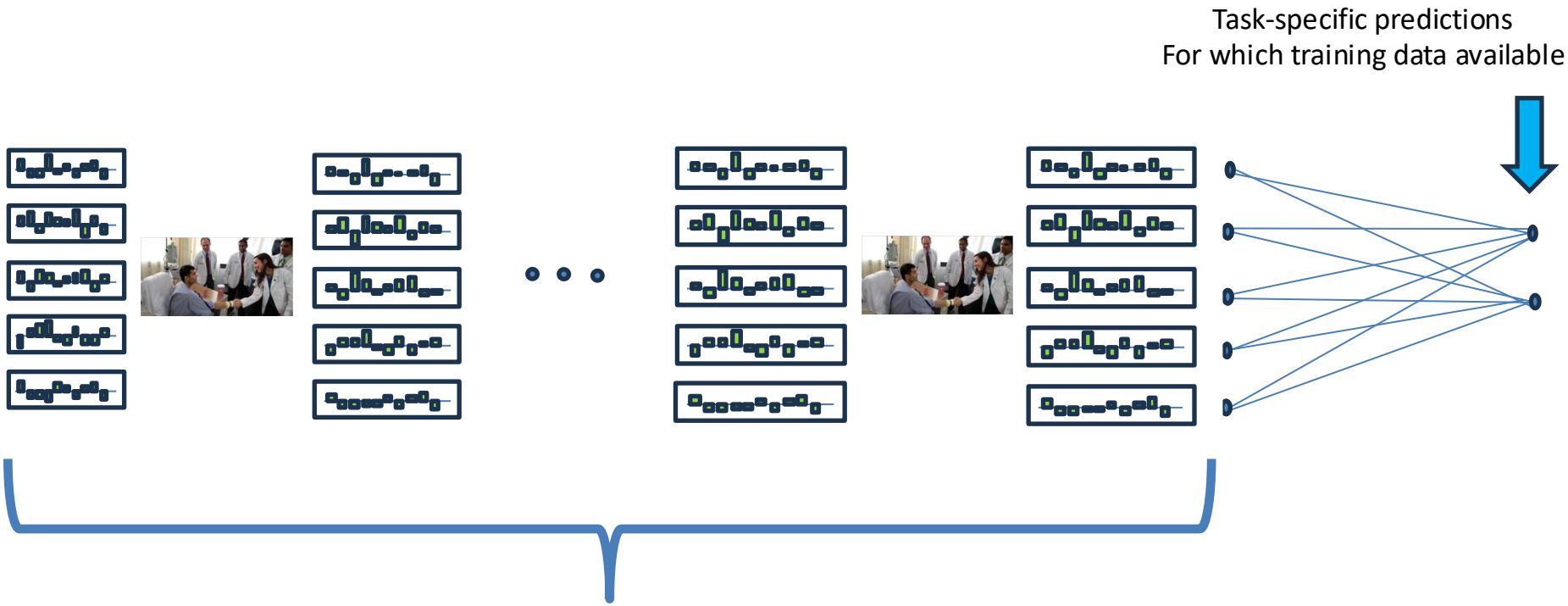
Context-independent vector for each word (“person”)

Predict next word (“person”)

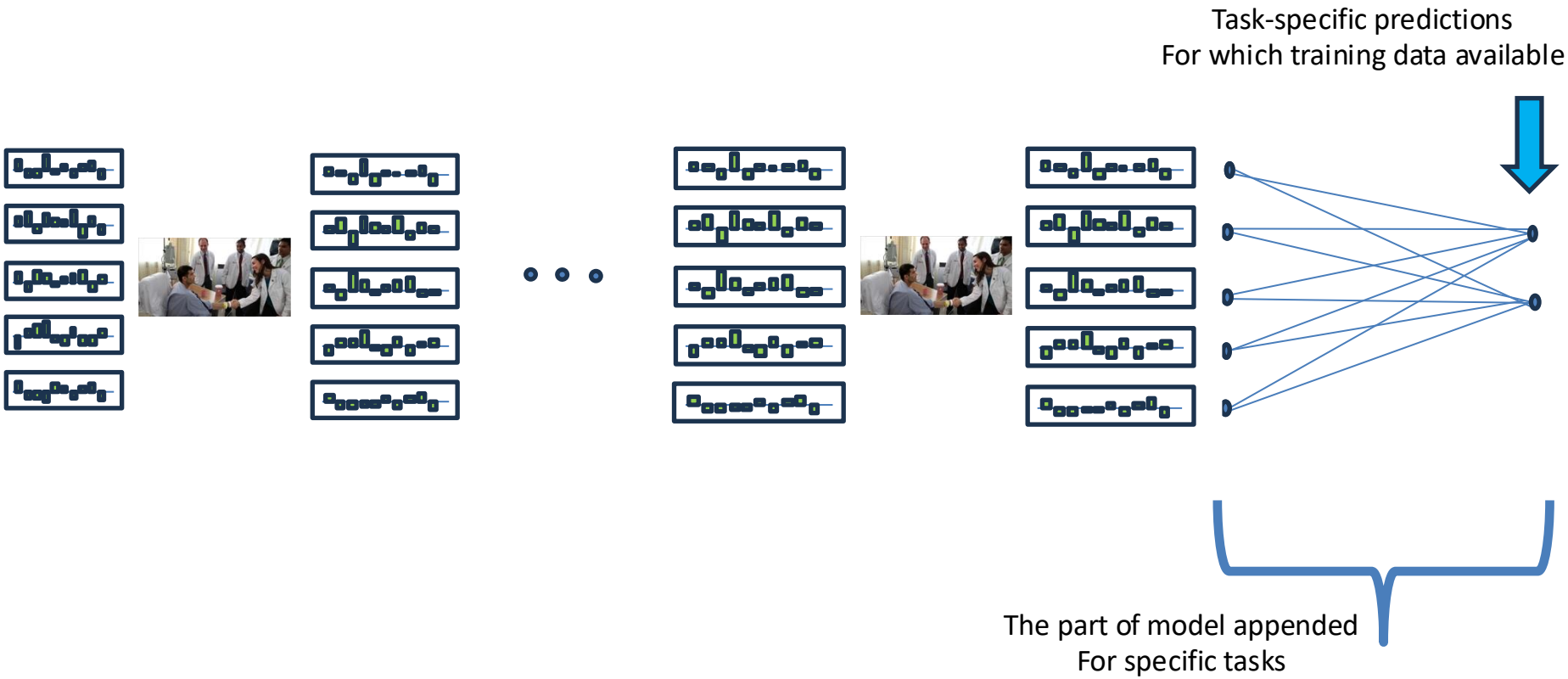
Fine Tuning



Fine Tuning


Task-specific predictions
For which training data available
Fine-tune/refine/update parameters from GPT for
desired specific prediction tasks

Fine Tuning


Task-specific predictions
For which training data available
The part of model appended
For specific tasks

Diving into Details of LLMs

Word to Vector (Word2Vec)

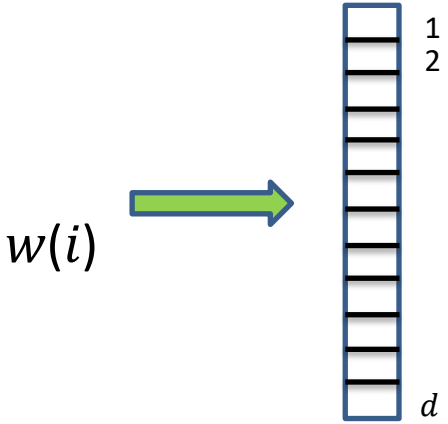
- The key to neural language models involves mapping each word to a vector
- Such word vectors are also called word “embeddings”
- Let $w(i)$ represent the i-th word in the vocabulary

Word to Vector (Word2Vec)

- The key to neural language models involves mapping each word to a vector

- Such word vectors are also called word "embeddings"

- Let $w(i)$ represent the i-th word in the vocabulary

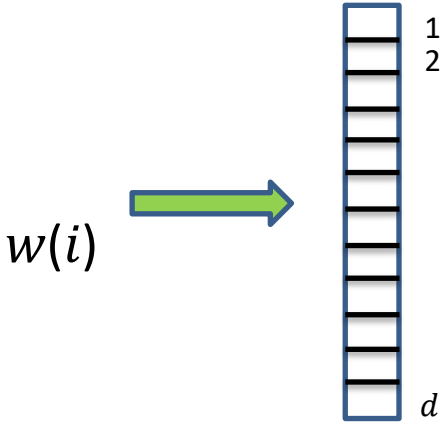


Word to Vector (Word2Vec)

- The key to neural language models involves mapping each word to a vector

- Such word vectors are also called word "embeddings"

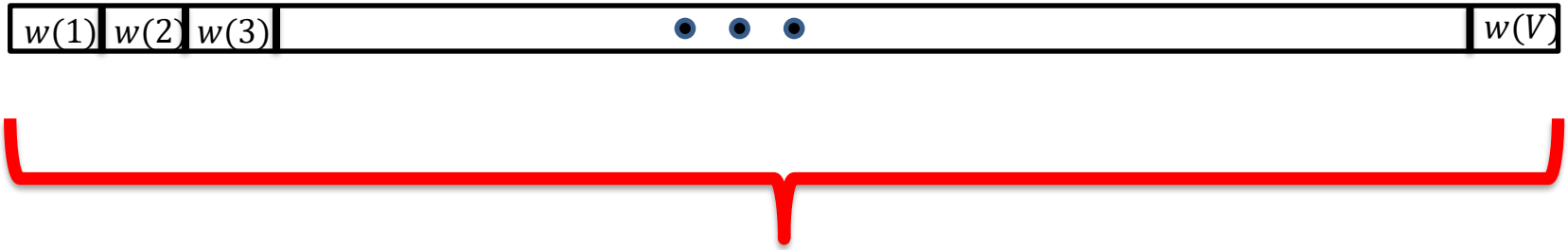
- Let $w(i)$ represent the i-th word in the vocabulary



Map each word to d-dimensional vector

Word to Vector (Word2Vec)

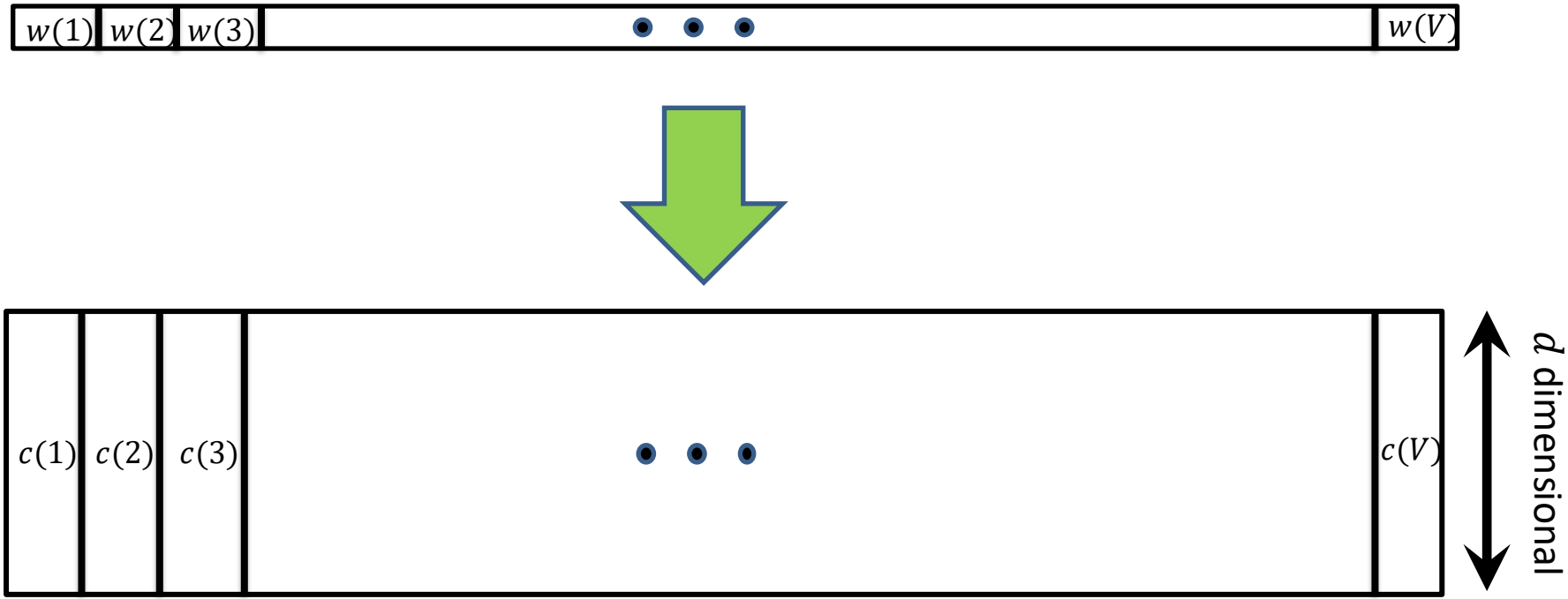
- The set of words in the vocabulary mapped to a codebook of vectors



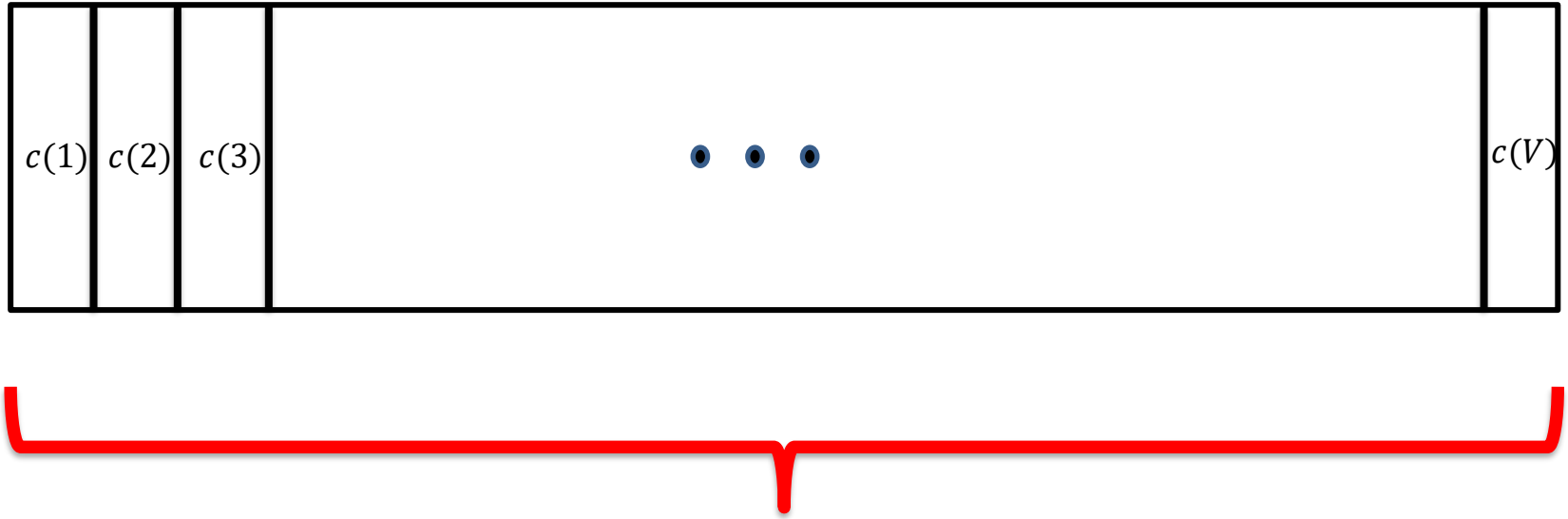
V words in Vocabulary

Word to Vector (Word2Vec)

- The set of words in the vocabulary mapped to a codebook of vectors



Word to Vector (Word2Vec)



Vocabulary Codebook

Learning Word2Vec Representations

- There are many ways to learn the codebook of vectors for each word in vocabulary

- Assume we have a massive corpus of unlabeled documents with which to learn

- **Key idea**: For each word in a document, we should be able to predict the presence of the surrounding words

Map Sentence to Sequence of Vectors

- Consider a sentence, composed of a sequence of N words

$$w_1 \quad w_2 \quad \dots \quad w_N$$

- These N words are represented by a sequence of codes, each of dimension d

Accounting for Word Context

- ❑ Words often have many meanings, depending on context
- ❑ Restrictive to map each word to a single vector
- ❑ Wish to extend this construct, to account for context of surrounding words
- ❑ Now vector associated with a given word may vary, based on context

Inner Product Between Word Vectors

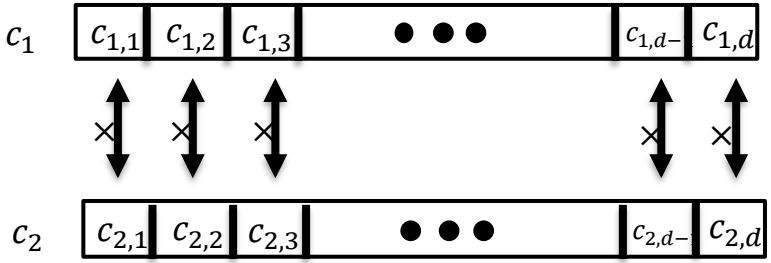
- Map the code for the first word, c_1, into a new code, that accounts for the surrounding N-1 other words

- Explicitly define code $c_1 = (c_{1,1}, c_{1,2}, c_{1,3}, \ldots, c_{1,d})$

Inner Product Between Word Codes

- Define the "inner product" between codes c_1 and c_2 as

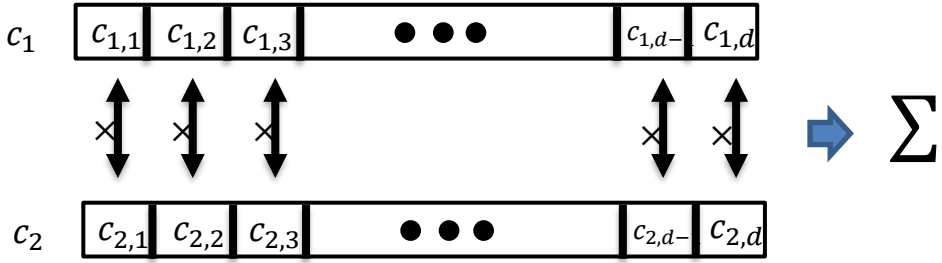
$$c_1 \cdot c_2 = (c_{1,1} \times c_{2,1}) + (c_{1,2} \times c_{2,2}) + (c_{1,3} \times c_{2,3}) + \cdots + (c_{1,d} \times c_{2,d}) = \sum_{i=1}^{d} c_{1,i} \times c_{2,i}$$



Sum the d results from the multiplications

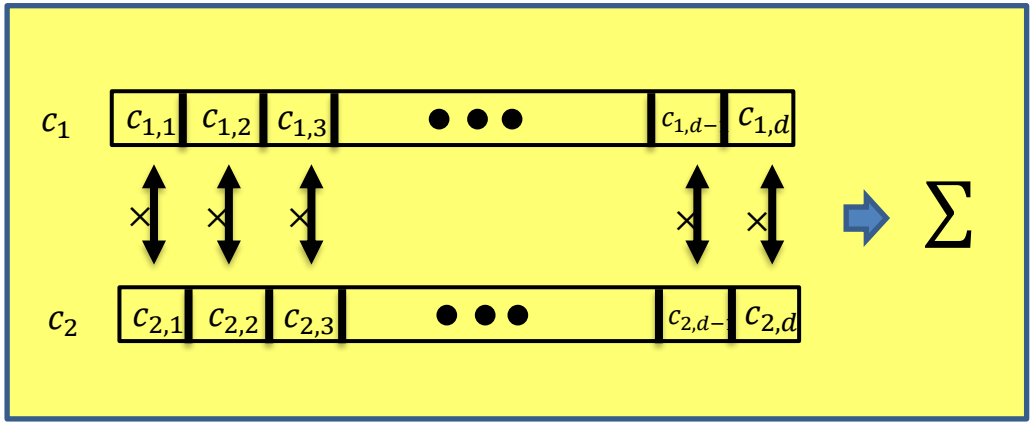
Meaning of Inner Product

$$c_1 \cdot c_2 = \sum_{i=1}^{d} c_{1,i} \times c_{2,i}$$

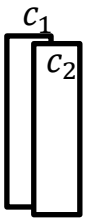


- If words w_1 and w_2 are similar, we expect the codes c_1 and c_2 to also be similar
- In this case the inner product $c_1 \cdot c_2$ will be large and positive
- If the words are dissimilar, the inner product will be small or negative

Concise Symbol of Inner Product



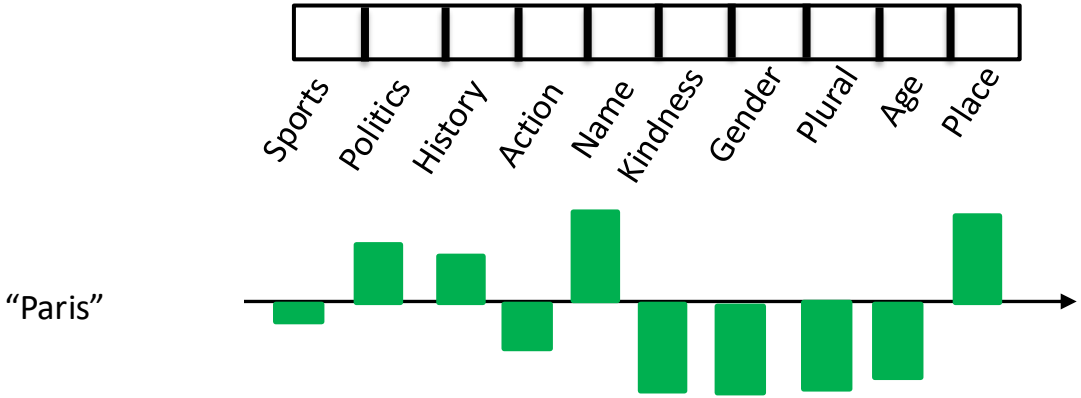
$=$



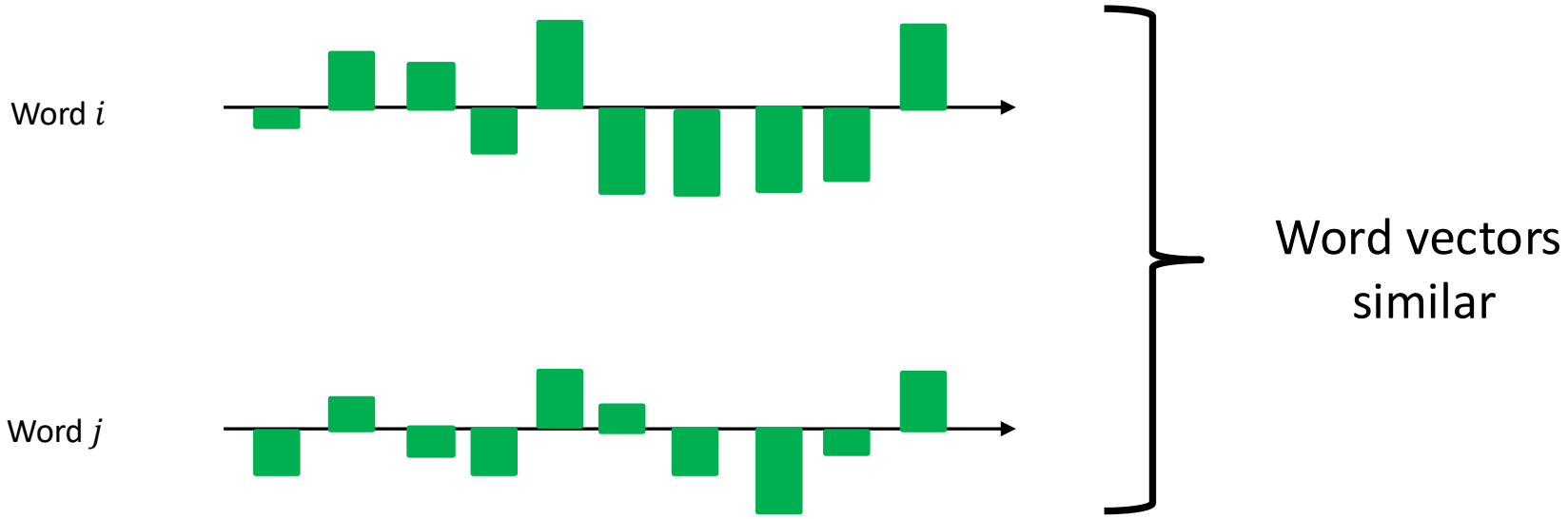
Meaning of Word Inner Product

- Recall the intuition behind word vectors

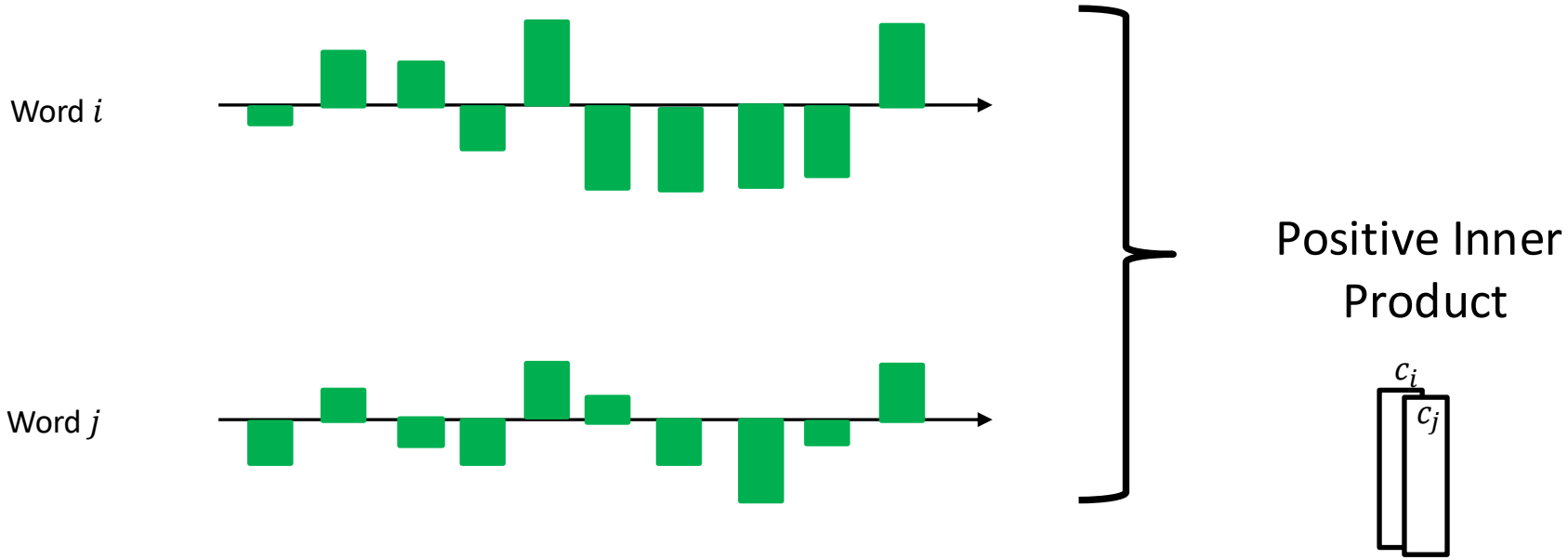
- Note the vector may be 256 dimensions, or larger (10-dimensional vectors illustrative)



Meaning of Word Inner Product

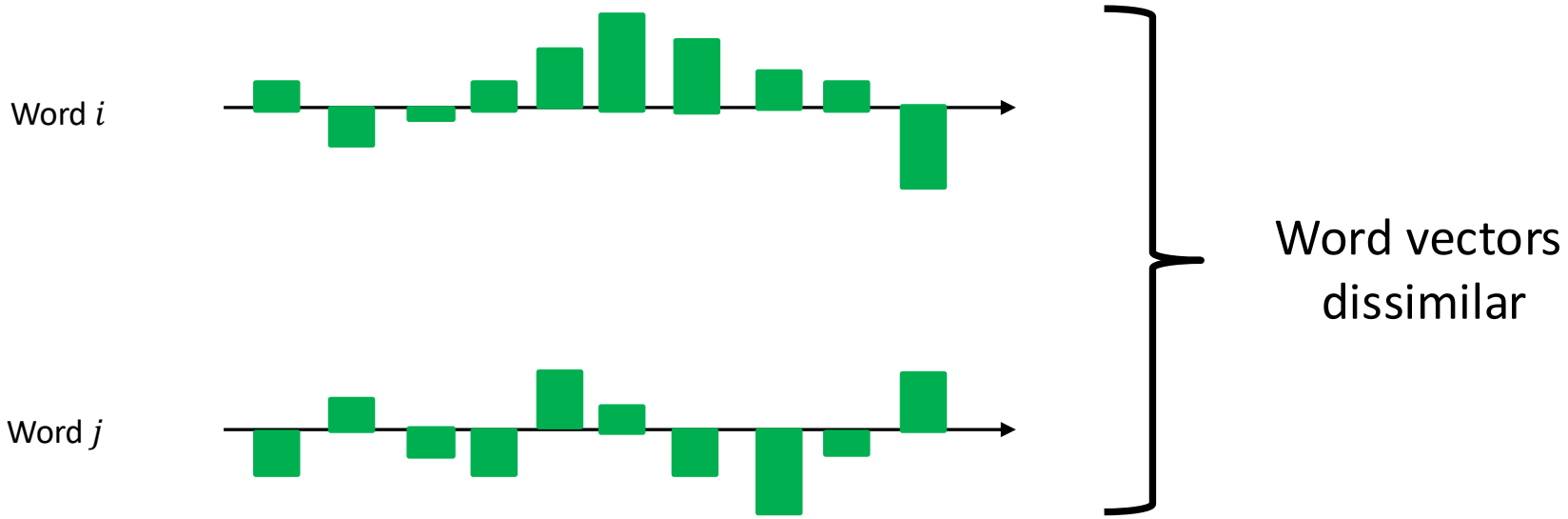


Meaning of Word Inner Product

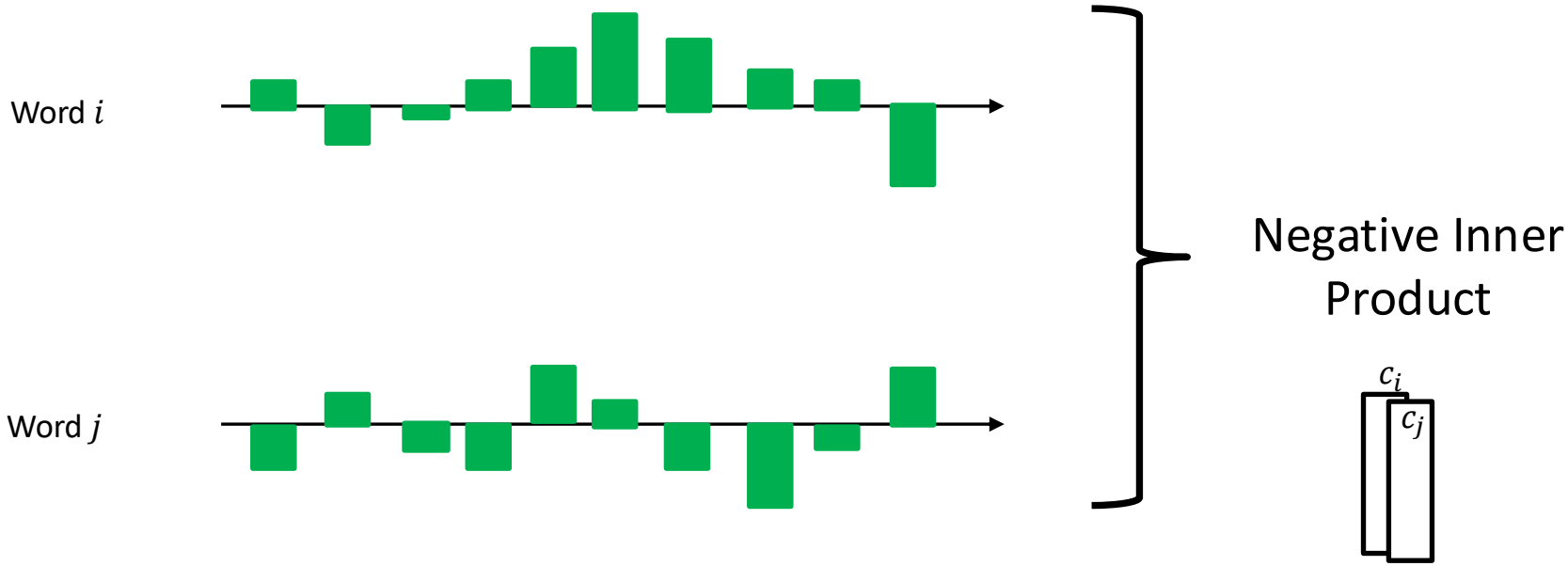


The more positive, the more the words are in alignment

Meaning of Word Inner Product

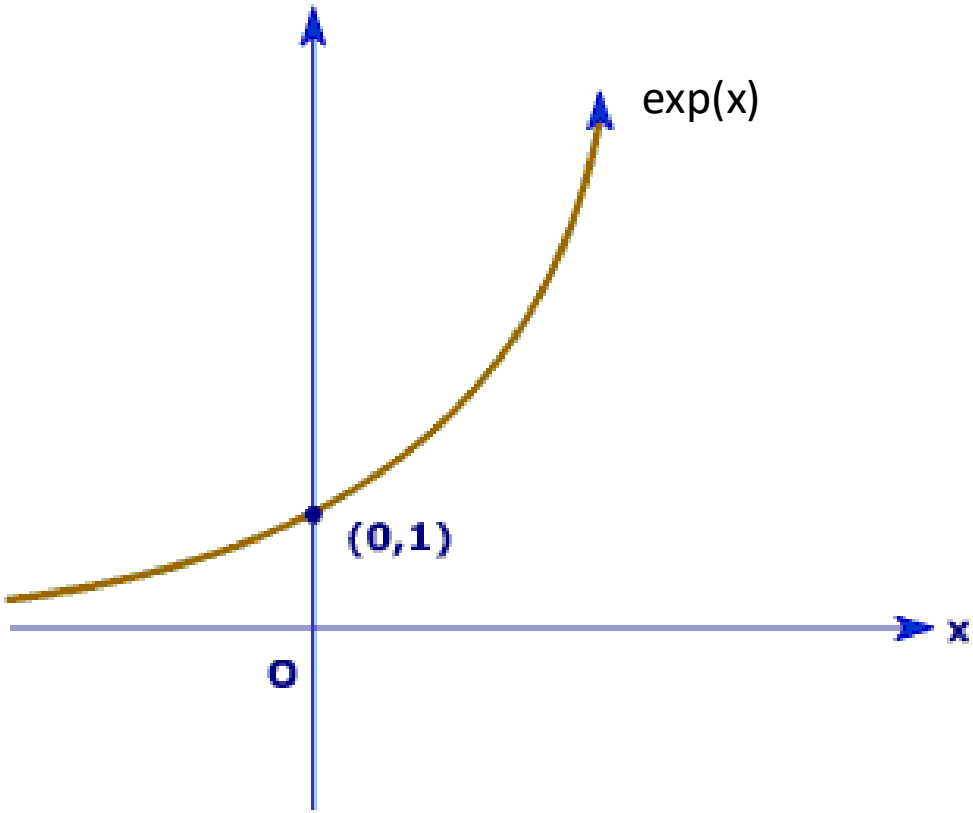


Meaning of Word Inner Product

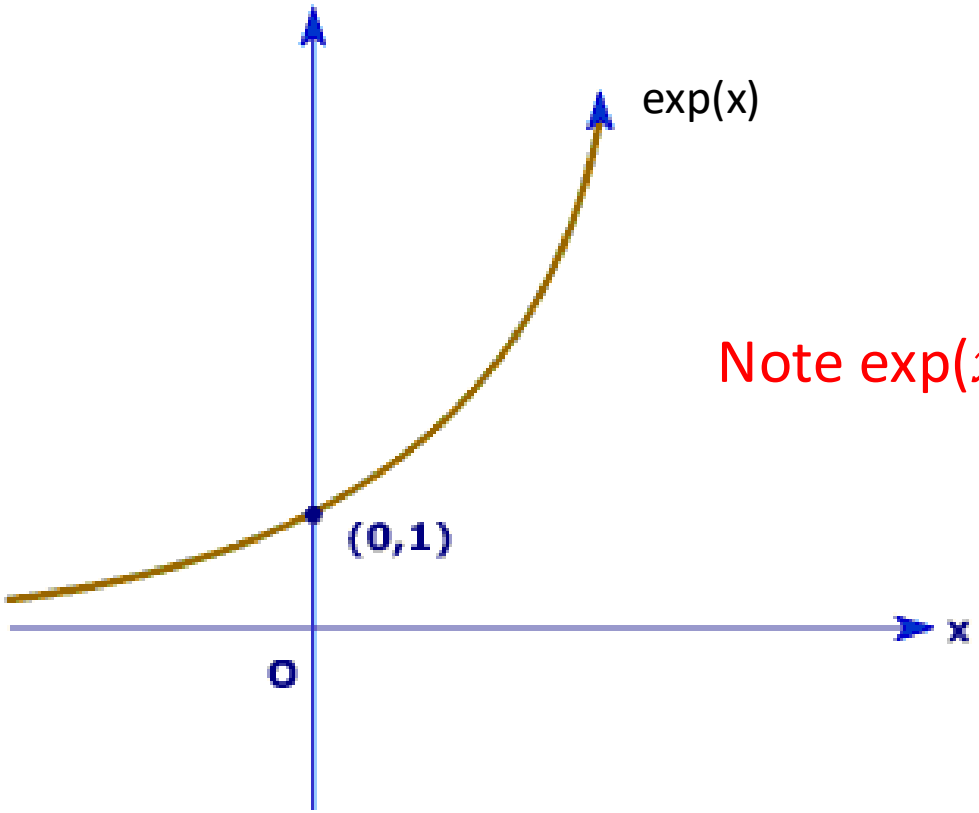


The more negative, the more the words are in opposite alignment

Exponential Function

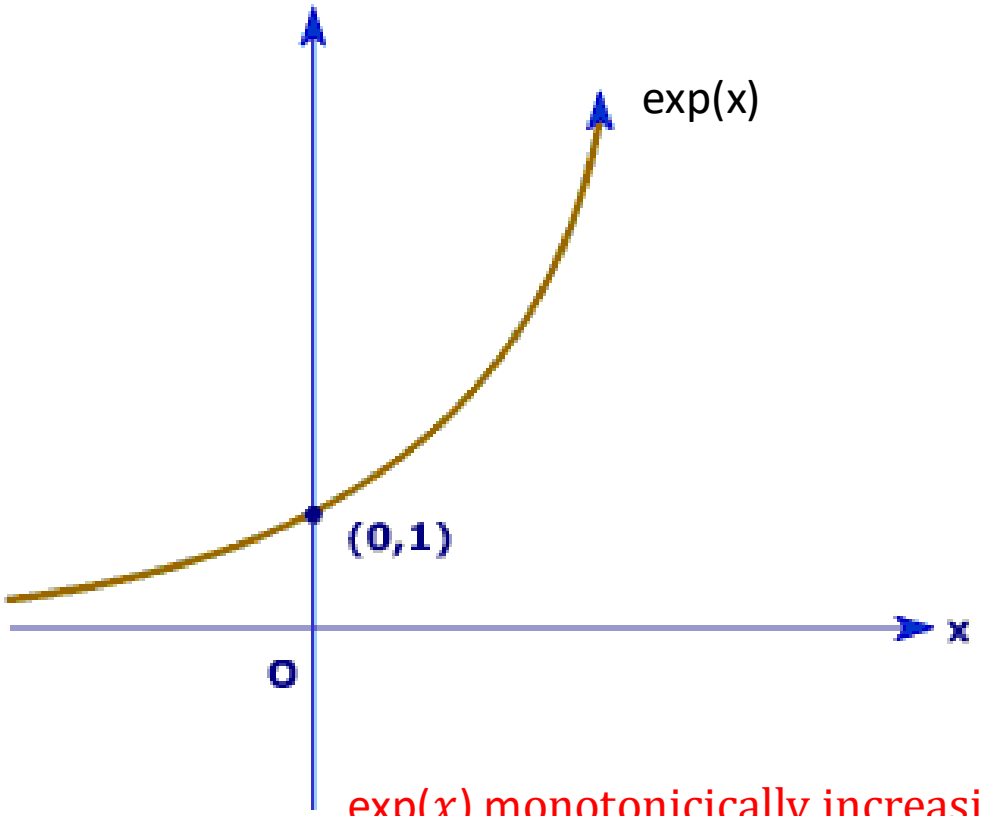


Exponential Function



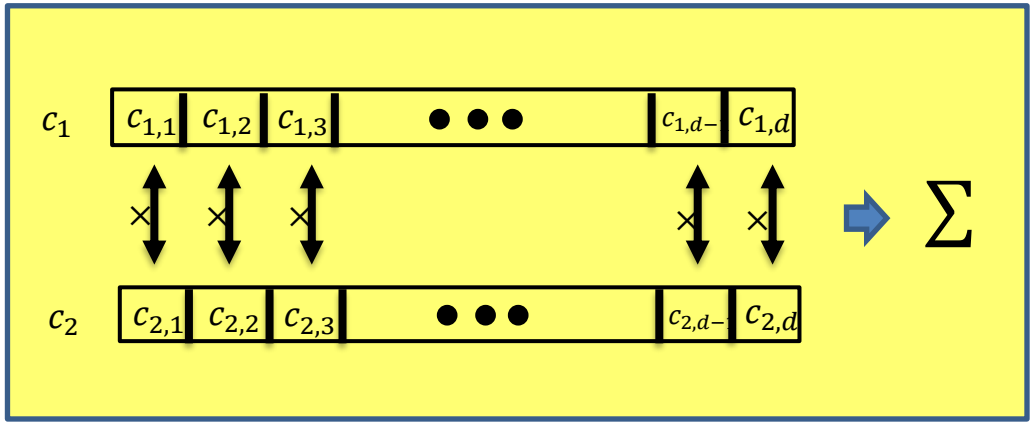
Note exp(x)>0 for all x

Exponential Function

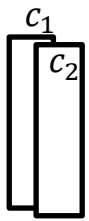


$\exp(x)$ monotonicically increasing with increasing x

Exponentiate Inner Product



$=$



- Larger $c_1 \cdot c_2$ yields larger $\exp(c_1 \cdot c_2)$

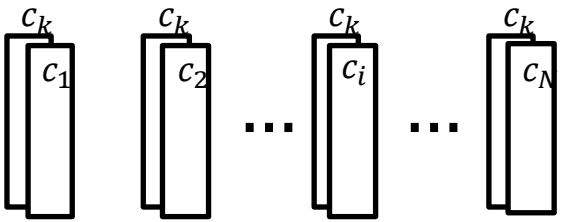
- $\exp(c_1 \cdot c_2)$ is always greater than zero

Quantify How Similar All Words are to Word k

$r_{k\to 1}$ $r_{k\to 2}$ ··· $r_{k\to i}$ ··· $r_{k\to N}$

$\exp(c_k \cdot c_1)$ $\exp(c_k \cdot c_2)$ ··· $\exp(c_k \cdot c_N)$

$c_k \cdot c_1$ $c_k \cdot c_2$ $c_k \cdot c_i$ $c_k \cdot c_N$



$$r_{k\to i} = \frac{\exp(c_k \cdot c_i)}{\exp(c_k \cdot c_1) + \exp(c_k \cdot c_2) + \exp(c_k \cdot c_3) + \cdots + \exp(c_k \cdot c_N)}$$

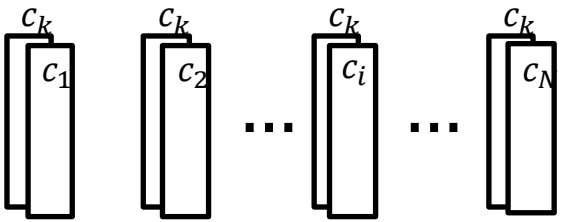
$$= \frac{\exp(c_k \cdot c_i)}{\sum_{n=1}^{N} \exp(c_k \cdot c_n)} \geq 0$$

Quantify How Similar All Words are to Word k

$r_{k\to 1}$ $r_{k\to 2}$ $r_{k\to i}$ $r_{k\to N}$

$\exp(c_k \cdot c_1)$ $\exp(c_k \cdot c_2)$ ▪▪▪ $\exp(c_k \cdot c_N)$

$c_k \cdot c_1$ $c_k \cdot c_2$ $c_k \cdot c_i$ $c_k \cdot c_N$

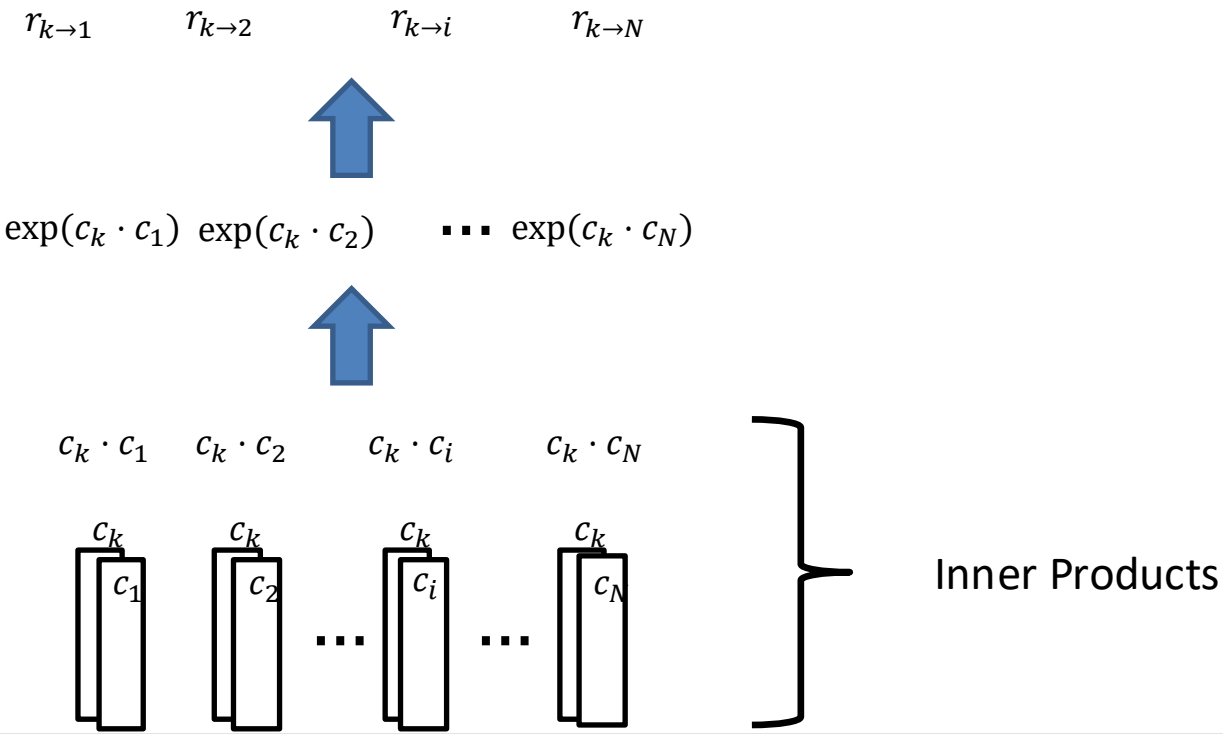


Relative similarity of word k to word i

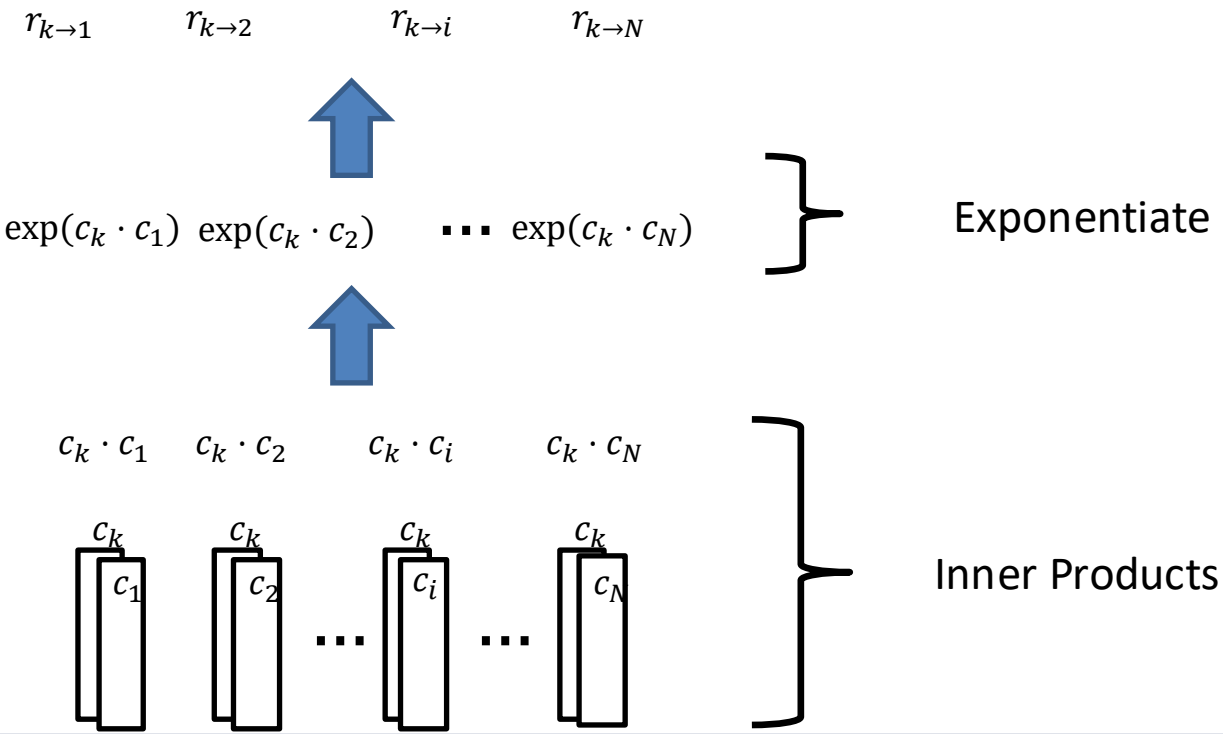
$$r_{k\to i} = \frac{\exp(c_k \cdot c_i)}{\exp(c_k \cdot c_1) + \exp(c_k \cdot c_2) + \exp(c_k \cdot c_3) + \cdots + \exp(c_k \cdot c_N)}$$

$$= \frac{\exp(c_k \cdot c_i)}{\sum_{n=1}^{N} \exp(c_k \cdot c_n)} \geq 0$$

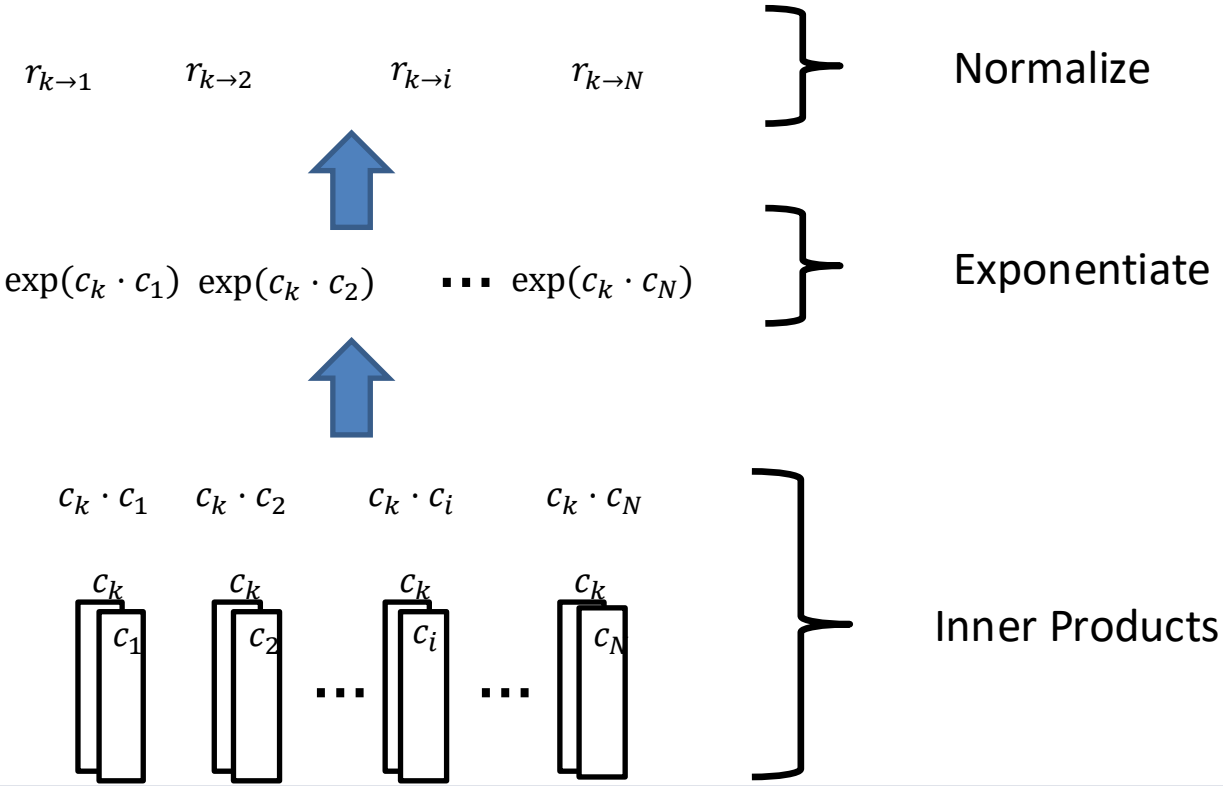
Quantify How Similar All Words are to Word *k*



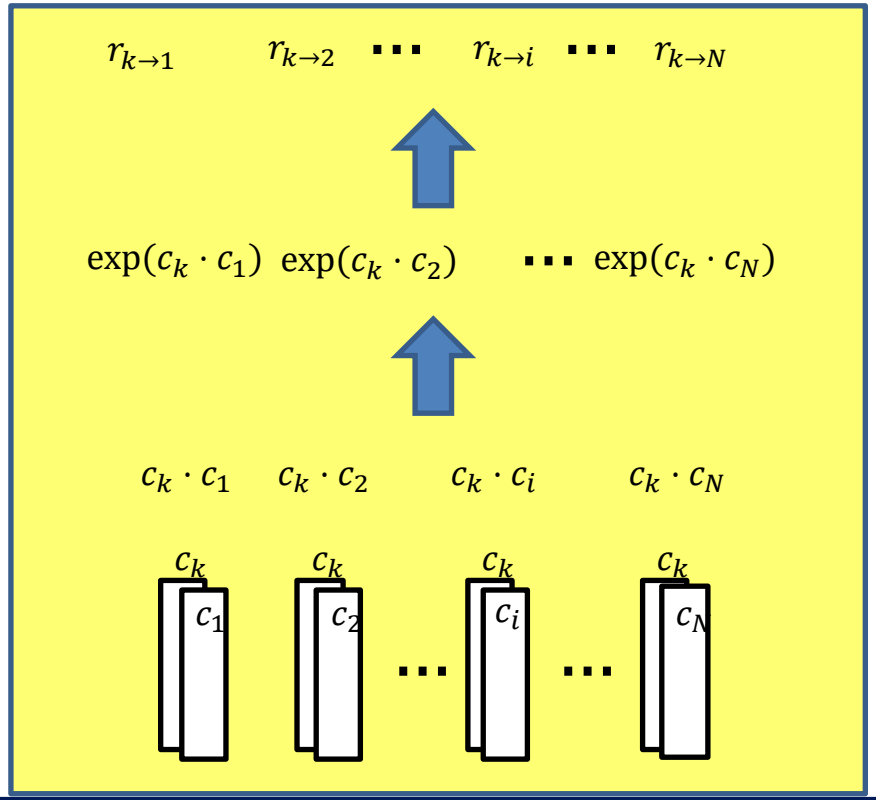
Quantify How Similar All Words are to Word *k*



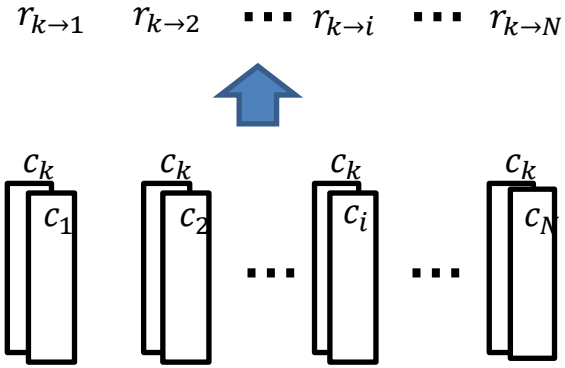
Quantify How Similar All Words are to Word *k*



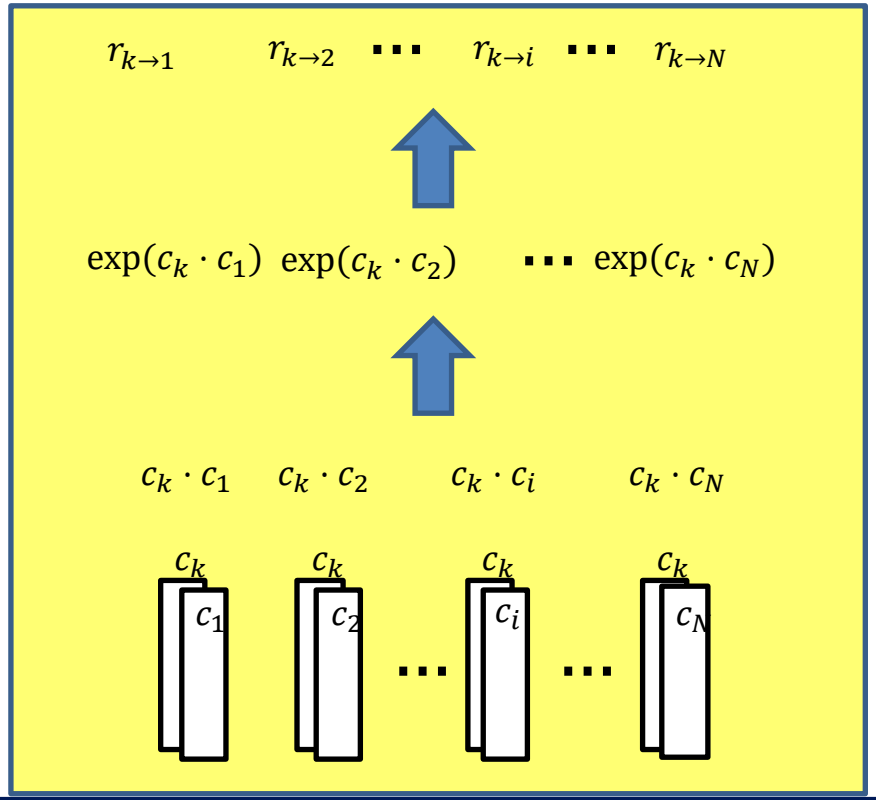
Simplify Depiction



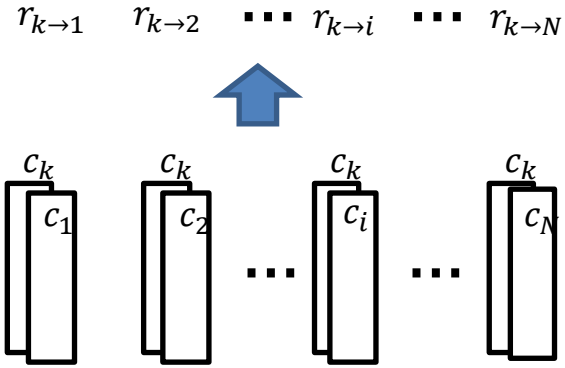
=



Simplify Depiction

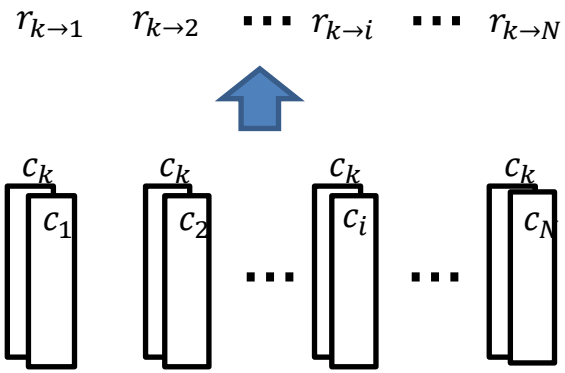


=



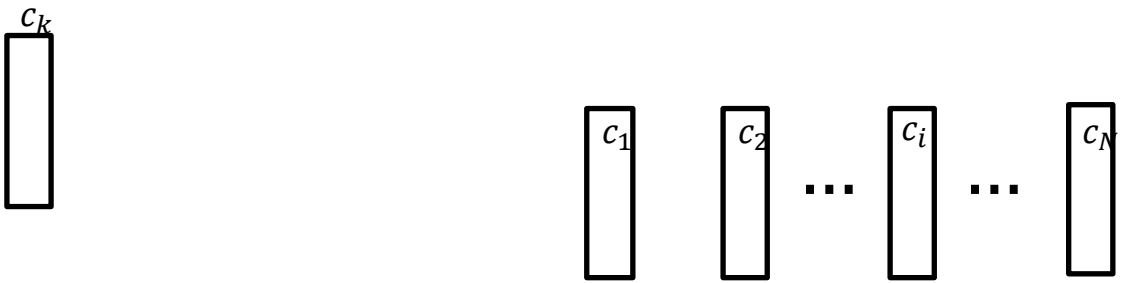
$$\sum_{i=1}^{N} r_{k\to i} = 1 \quad , \qquad r_{k\to i} > 0$$

Interpretation

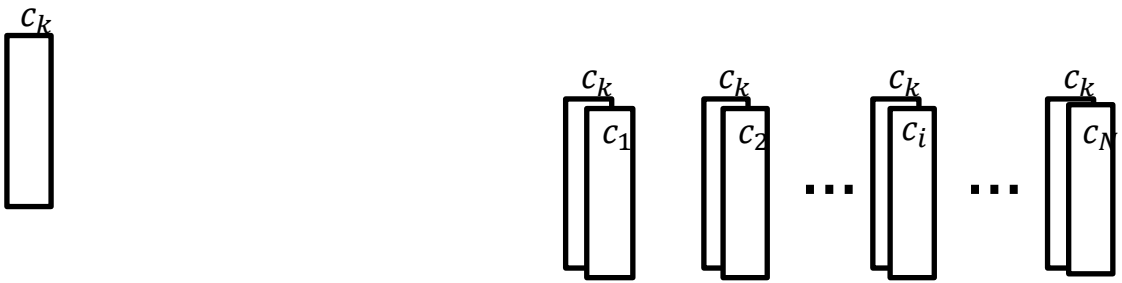


- The expression $r_{k\to i}$ quantifies how similar word k is to word i
- The larger $r_{k\to i}$ the larger the degree of correlation
- Will <u>learn</u> word vectors, so that their relative relationships (inner products) are consistent with natural language
- Perform (stochastic) gradient descent on word vectors
- Note that there is significant opportunity for parallel computing

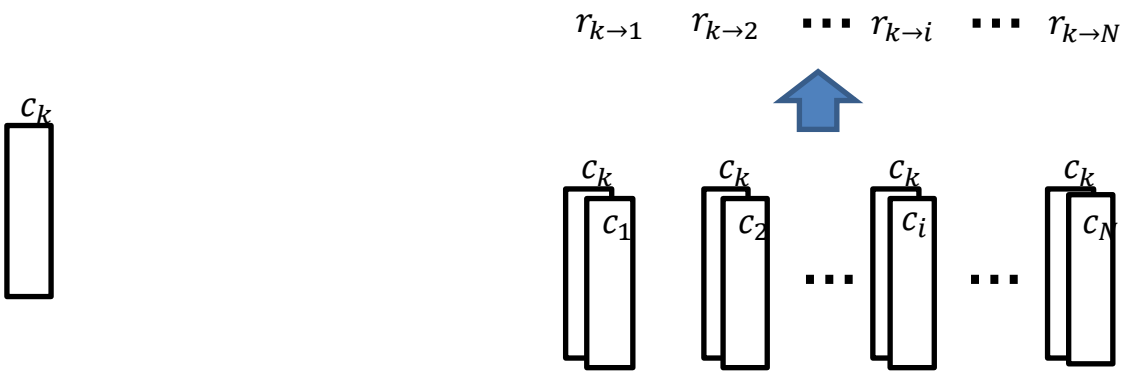
Map Original Sequence of Codes to New Code Sequence



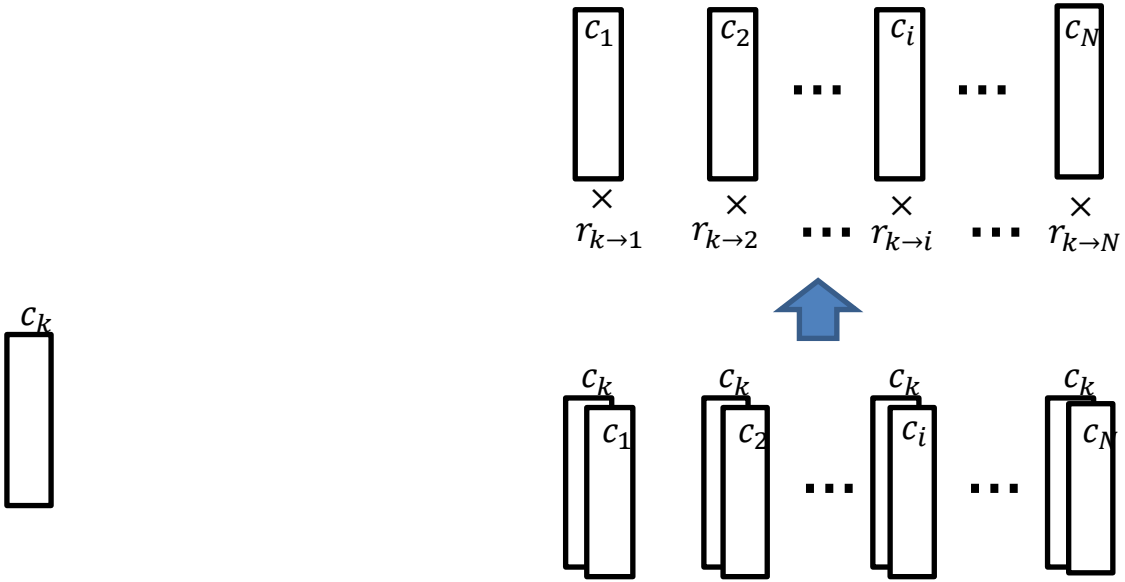
Map Original Sequence of Codes to New Code Sequence



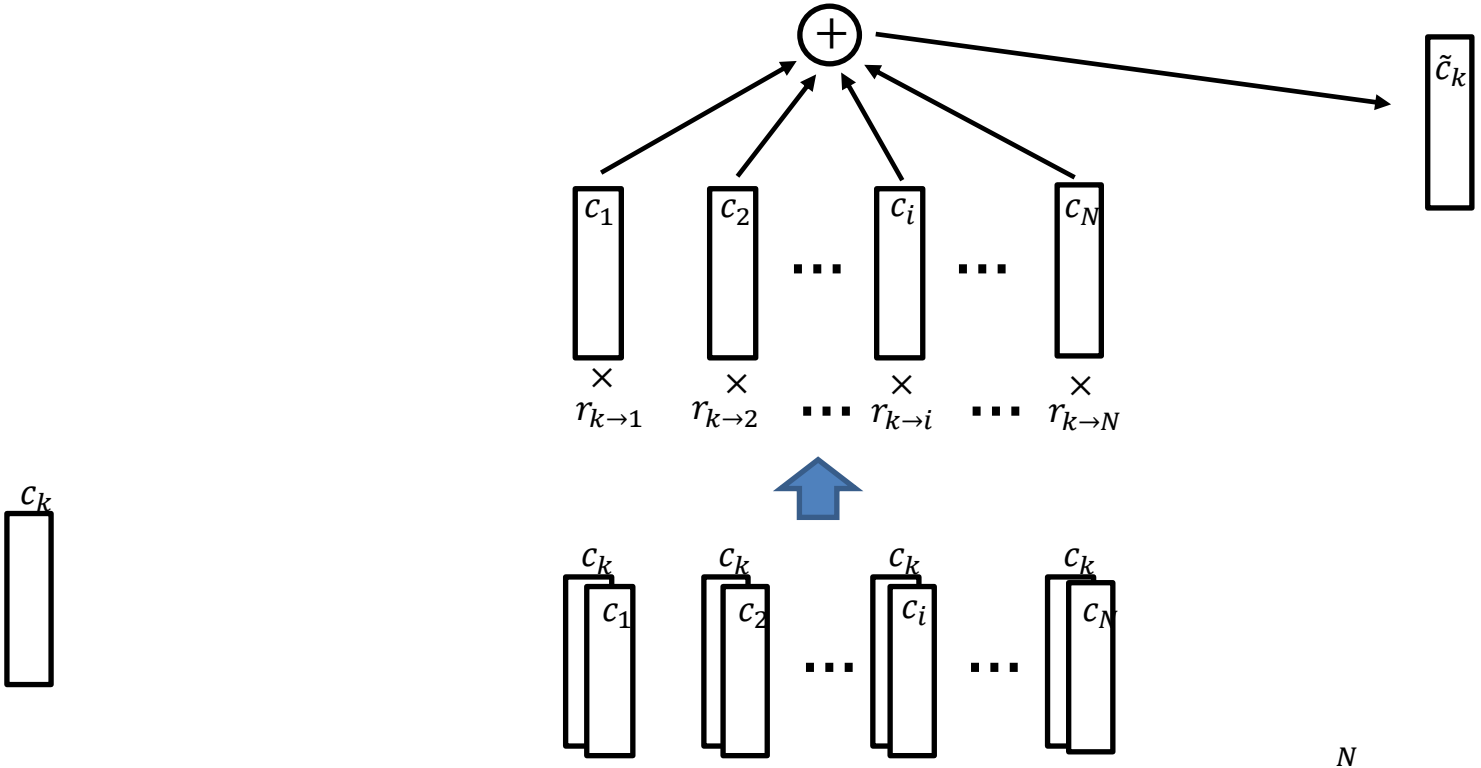
Map Original Sequence of Codes to New Code Sequence



Map Original Sequence of Codes to New Code Sequence

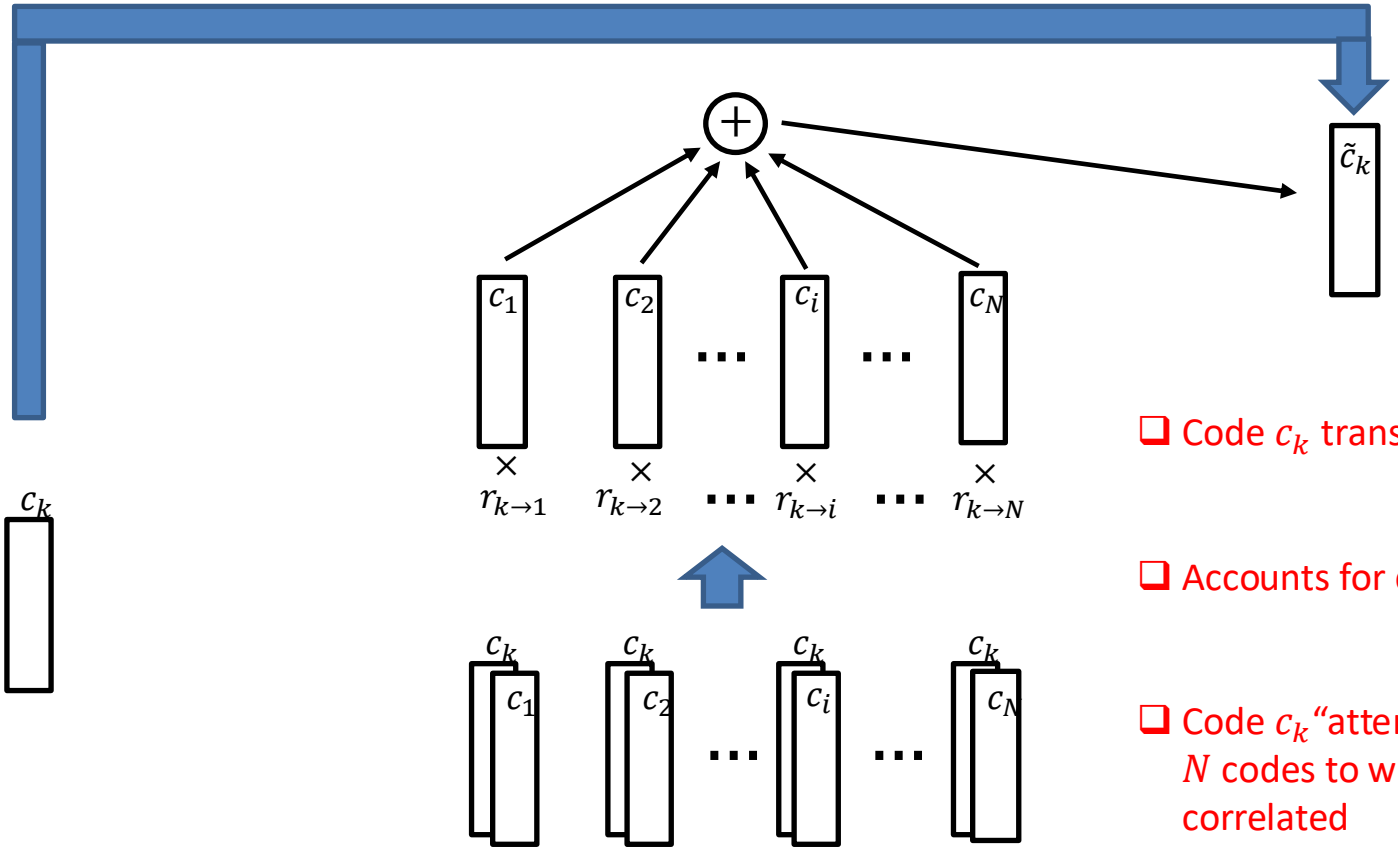


Map Original Sequence of Codes to New Code Sequence



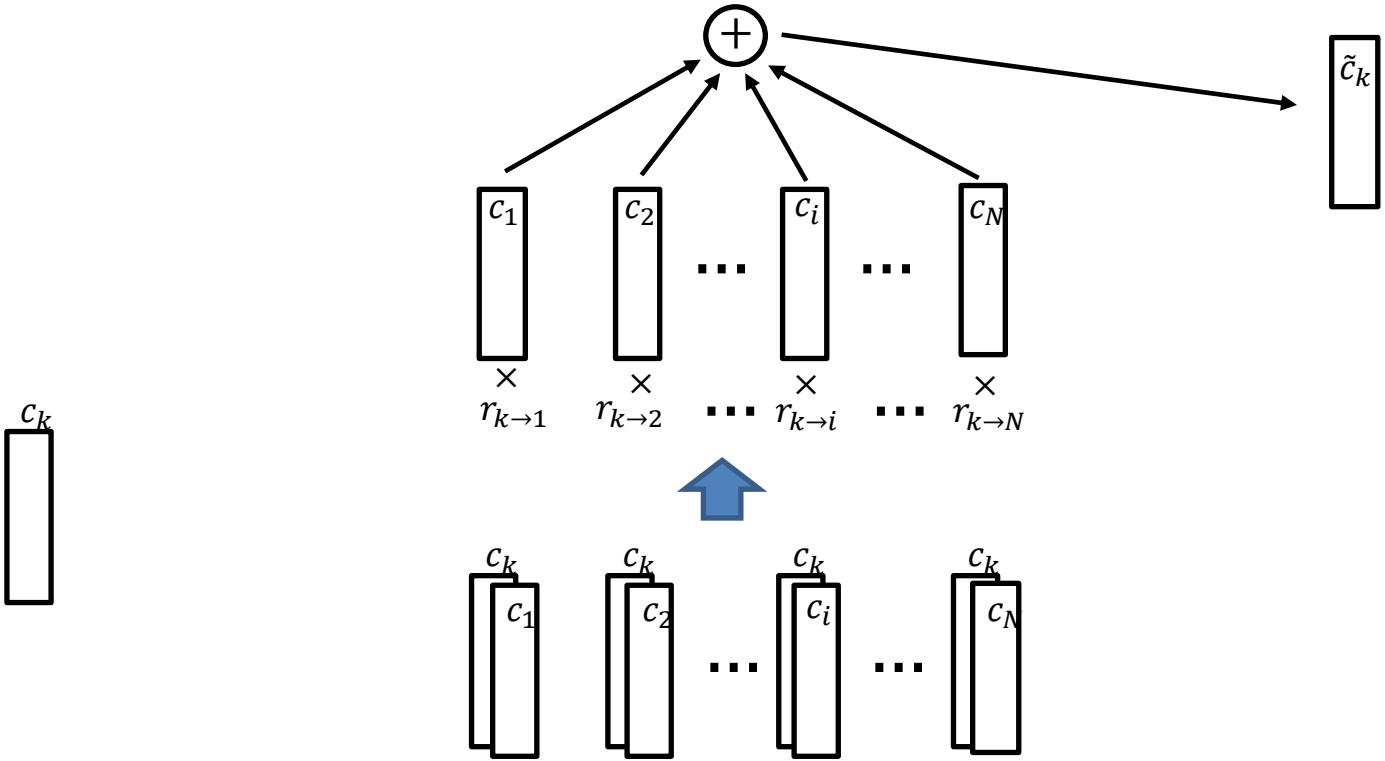
$$\tilde{c}_k = (r_{k\to1}\times c_1) + (r_{k\to2}\times c_2) + (r_{k\to3}\times c_3) + \cdots + (r_{k\to N}\times c_N) = \sum_{n=1}^{N} r_{k\to n}\times c_n$$

Vector Transformed with Context



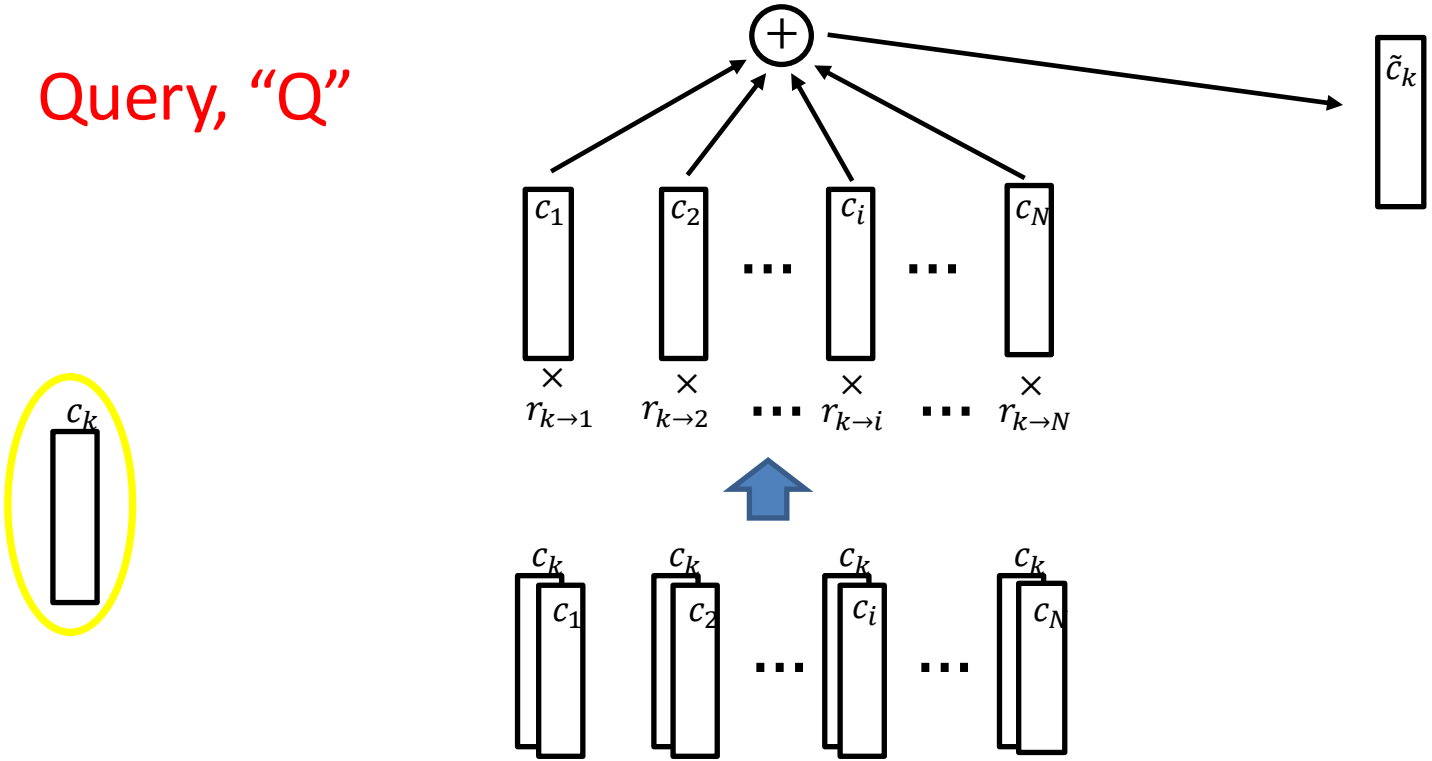
- Code c_k transformed to $\tilde{c}_k$
- Accounts for context of $\{c_1, \ldots, c_N\}$
- Code c_k "attends" to those of the N codes to which it is most correlated

Notation


$\tilde{c}_k$
c_1
c_2
c_i
c_N
$\times$
$r_{k\to 1}$
$r_{k\to 2}$
$r_{k\to i}$
$r_{k\to N}$
c_k
c_k
c_1
c_k
c_2
c_k
c_i
c_k
c_N

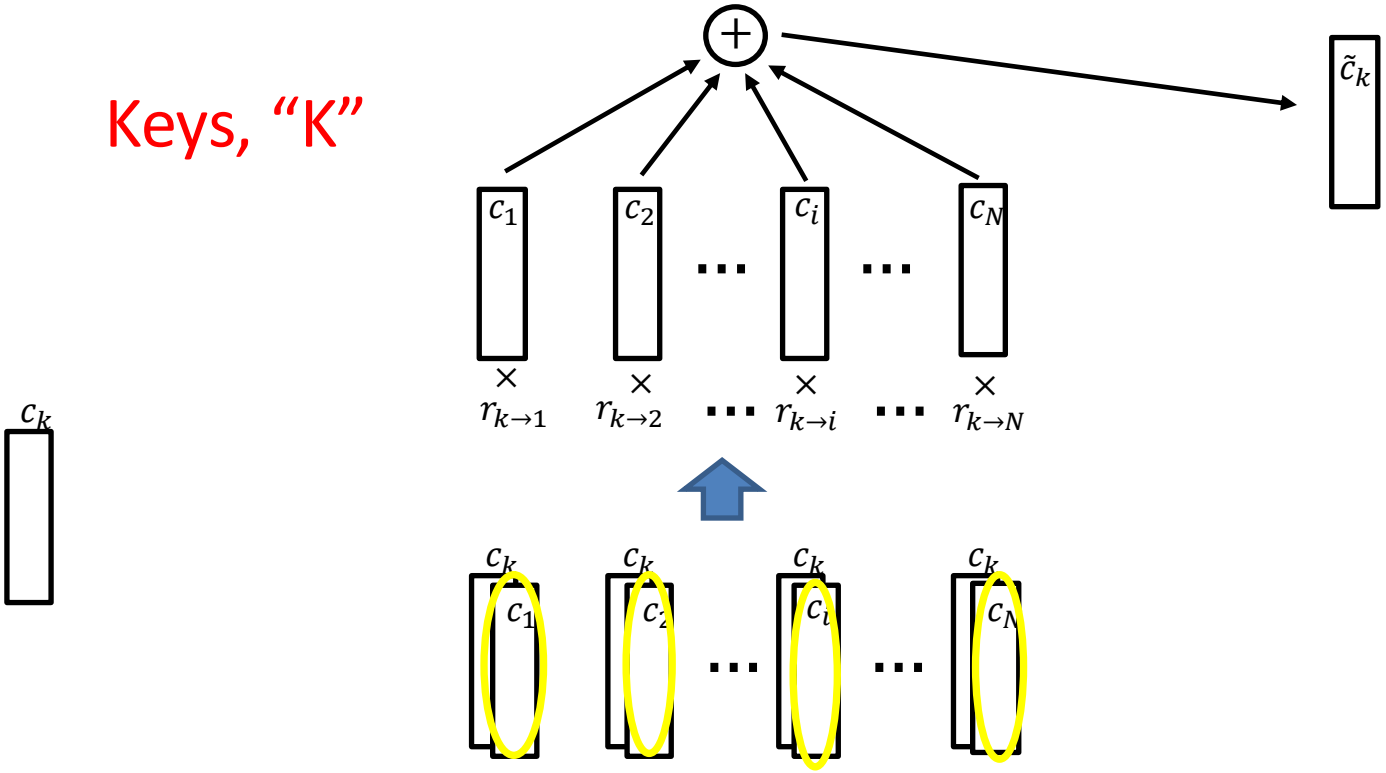
Notation

Query, "Q"



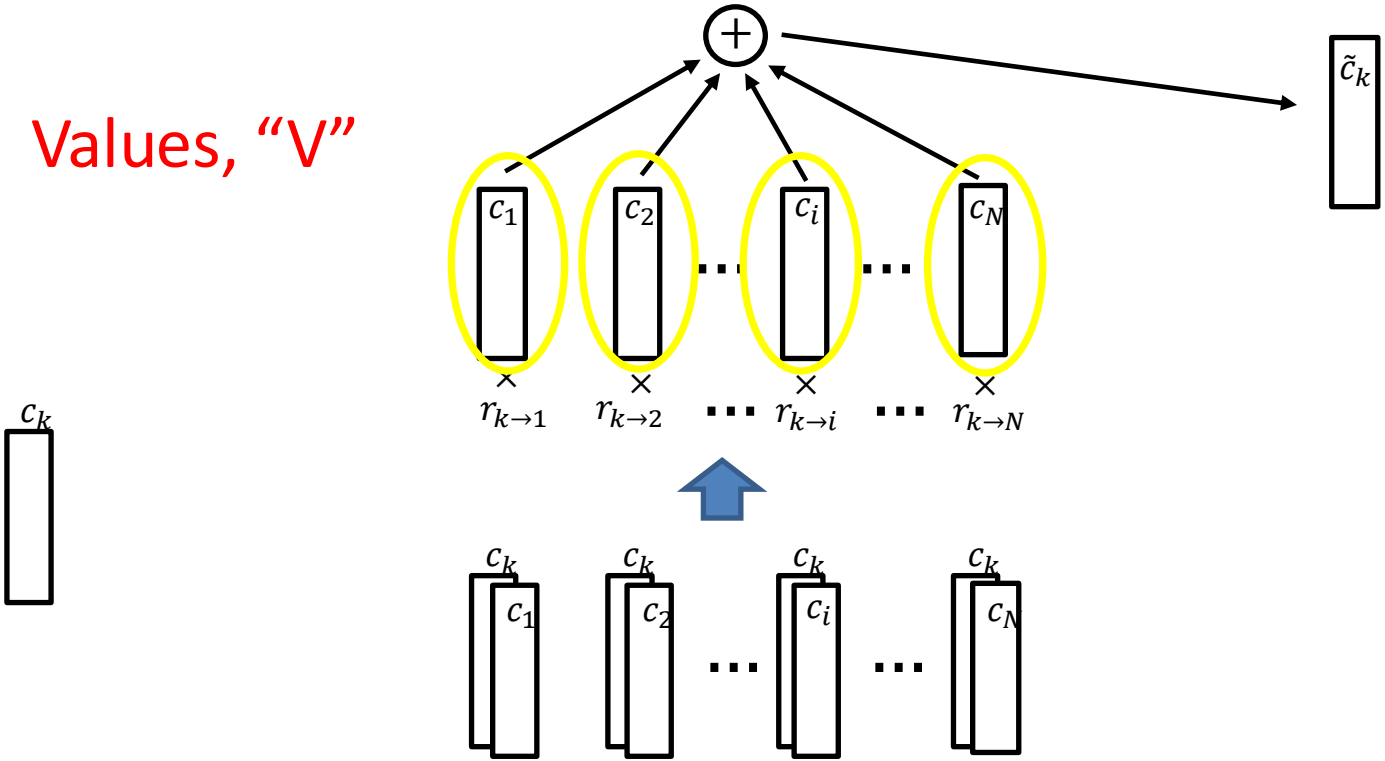
Notation

Keys, "K"

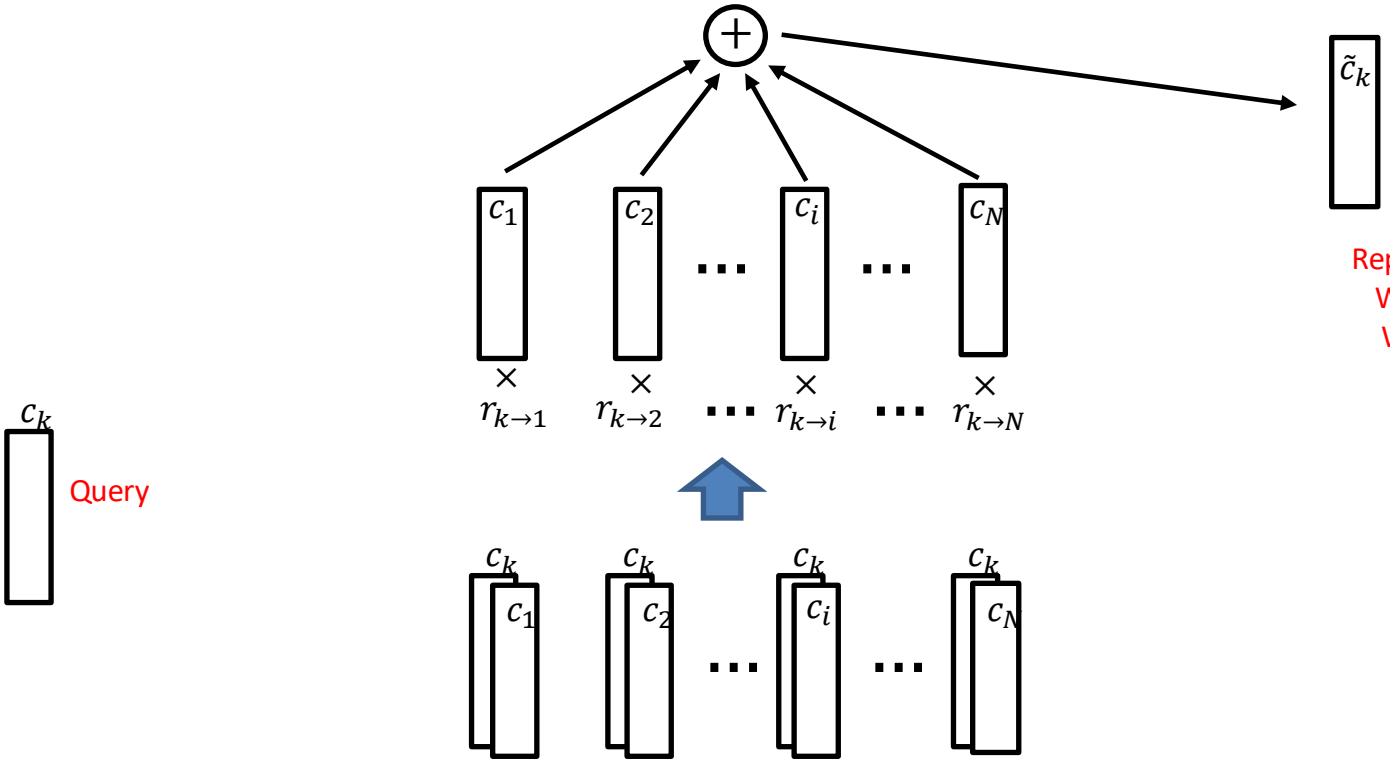


Notation

Values, “V”

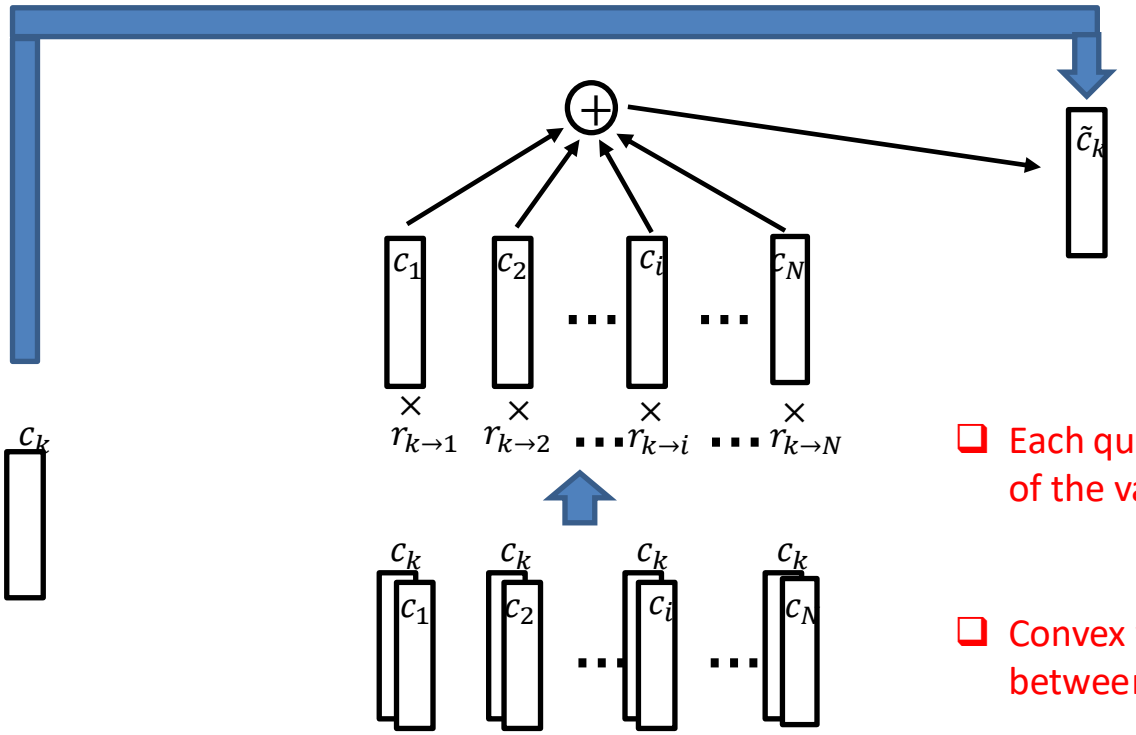


Meaning



Represent Query in Terms of Weighted Sum of Values, With Weight Defined by Match Between Query and Keys

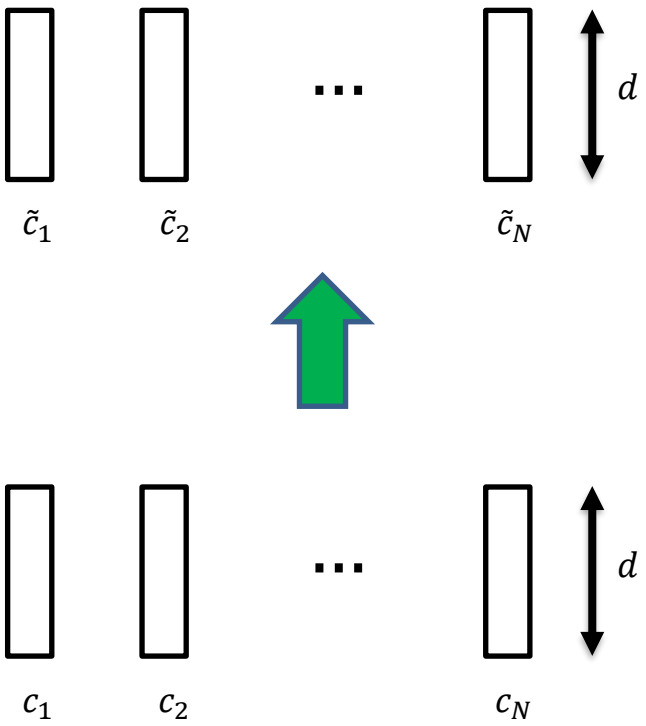
Vector Transformed with Context



- Each query c_k mapped to a "convex" combination of the values
- Convex weights given by match (inner product) between query and the keys
- Relative weights $r_{k\to i}$ define attention mechanism

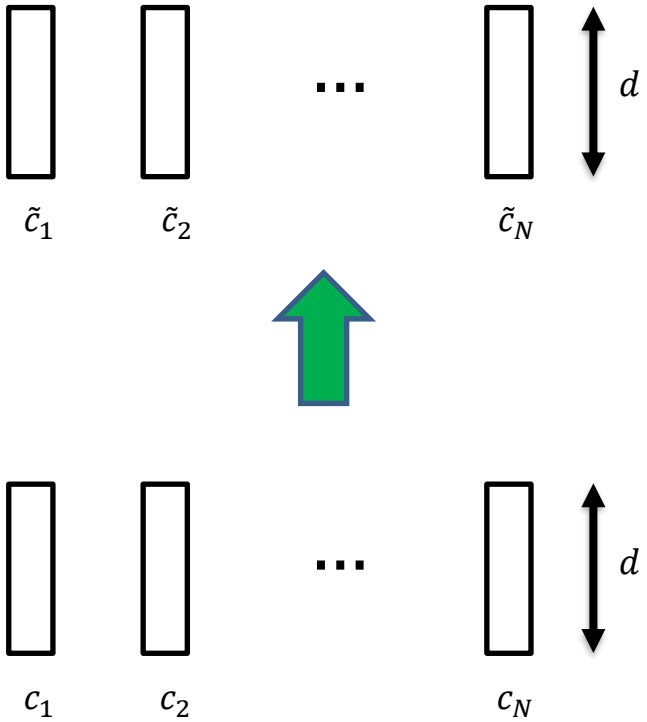
Self-Attention: Apply Attention to All Words, Relative to All Surrounding Words

- The Keys and Values correspond to input vectors
- Each output vector corresponds to one query
- Cycle through consideration of each input vector as the query vector

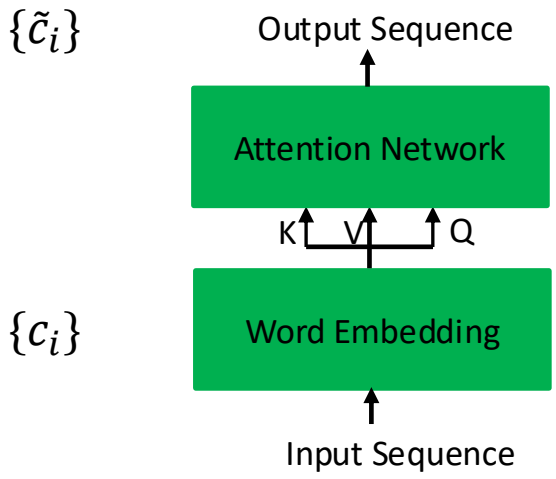


Interpretation

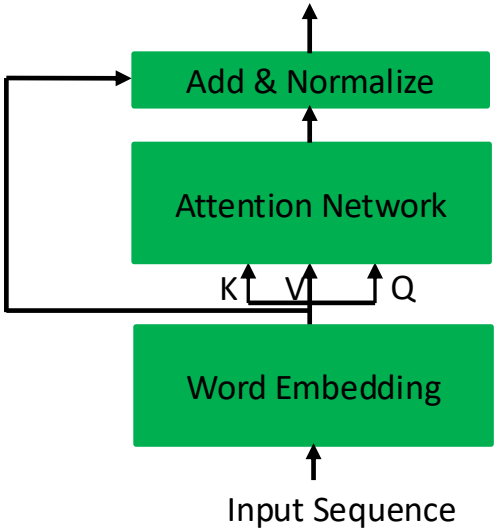
- The original codes $c_1, c_2, \ldots, c_N$ correspond to isolated words
- The updated codes $\tilde{c}_1, \tilde{c}_2, \ldots, \tilde{c}_N$ account for context of surrounding words
- However, <u>doesn't</u> account for ordering of input sequence
- Will introduce a method below to account for word order



Self-Attention Block Diagram

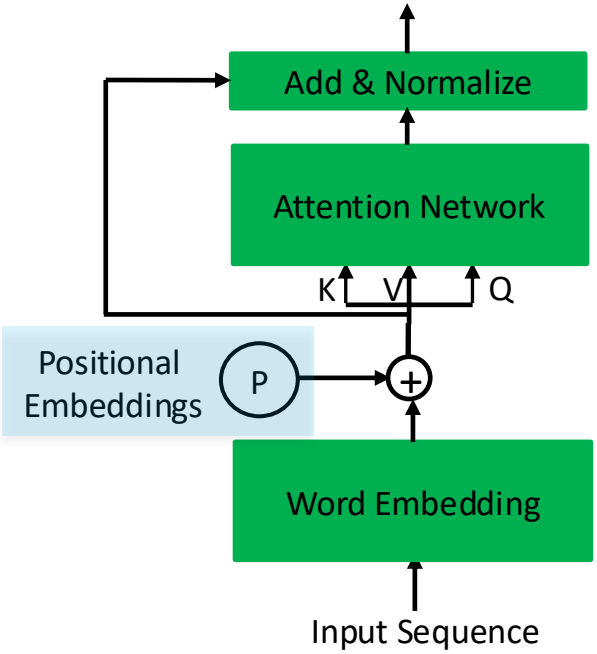


Modification 1: Introduce Skip Connections



Preserves Information in
Original Input Embeddings
+
Accounts for Context

Modification 2: Positional Embeddings



Accounts for Word Order

Positional Embeddings

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

- Dimension i of the positional encoding corresponds to a sinusoid, with frequency $\omega_{2i} = 1/(10000)^{2i/d}$

- One of many ways the positions may be encoded

- Each position has a unique code (embedding), added to the corresponding word embedding

Positional Embeddings

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

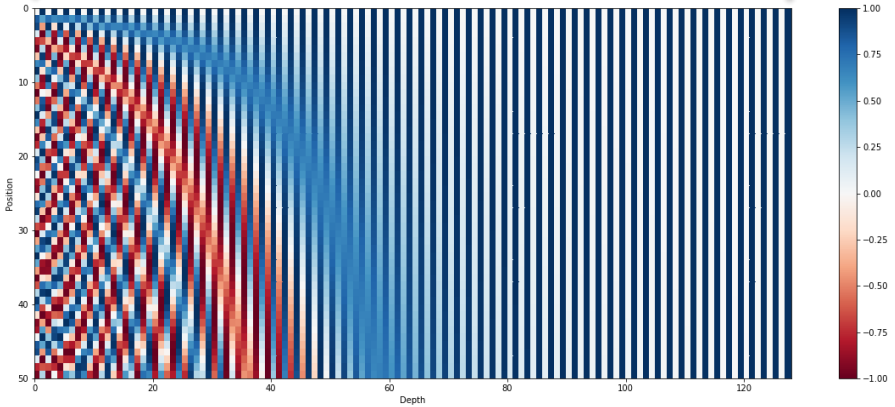
Vector representing integer

Inefficient: 4 dimensions to represent 2^4=16 values

Can we use soft bits?

Positional Embeddings

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1



Vector representing integer

Inefficient: 4 dimensions to represent 2^4=16 values

Can we use soft bits?

Periodic Functions (sin, cos)

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Positional Embeddings

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

$pos = 1, 2, \ldots.$ (word position)

- Dimension i of the positional encoding corresponds to a sinusoid, with frequency $\omega_{2i} = 1/(10000)^{2i/d}$

- One of many ways the positions may be encoded

- Each position has a unique code (embedding), added to the corresponding word embedding

Positional Embeddings

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

$i = 1, 2, \ldots, d$ (index of PE vector component)

- Dimension i of the positional encoding corresponds to a sinusoid, with frequency $\omega_{2i} = 1/(10000)^{2i/d}$

- One of many ways the positions may be encoded

- Each position has a unique code (embedding), added to the corresponding word embedding

Positional Embeddings

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

$pos = 1, 2, \ldots.$ (word position)

$i = 1, 2, \ldots, d$ (index of PE vector component)

- Dimension i of the positional encoding corresponds to a sinusoid, with frequency $\omega_{2i} = 1/(10000)^{2i/d}$

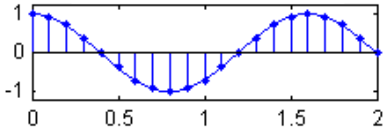
- One of many ways the positions may be encoded

- Each position has a unique code (embedding), added to the corresponding word embedding

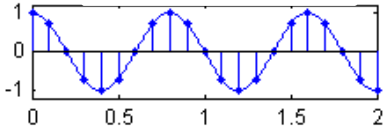
Positional Embeddings

Word Position

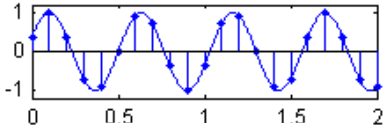
Embedding Dimension 1



Embedding Dimension 2



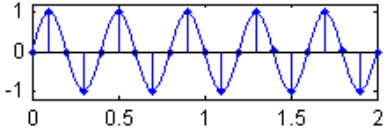
Embedding Dimension 3

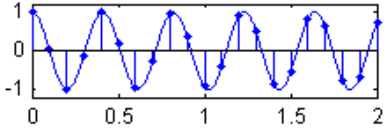


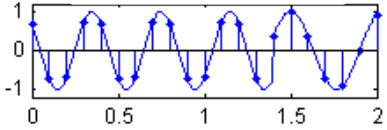
.

.

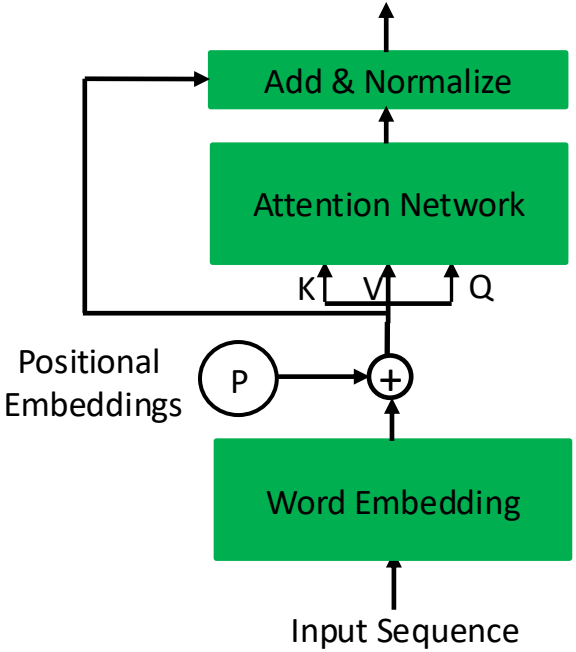
.







$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d})$$

One Attention Layer